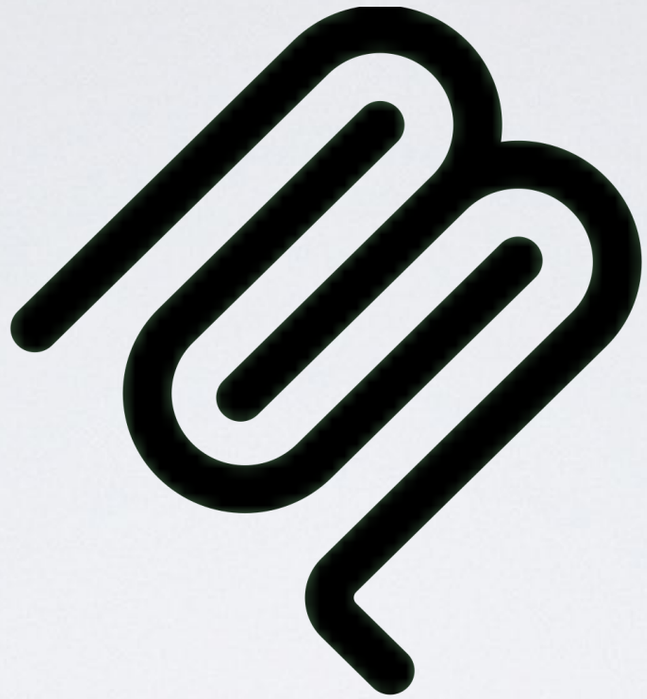


Model Context Protocol (MCP): The Future of User-Interaction?

ABOUT ME

Jonas Bandi
jonas.bandi@ivorycode.com
https://www.linkedin.com/in/jonasbandi



- Freelancer, in den letzten 12 Jahren vor allem in Projekten im Spannungsfeld zwischen modernen Webentwicklung und traditionellen Geschäftsanwendungen.
- Dozent an der Berner Fachhochschule seit 2007
- In-House Kurse & Beratungen zu Web-Technologien im Enterprise: UBS, Postfinance, Mobiliar, AXA, BIT, SBB, Elca, Adnovum, BSI ...

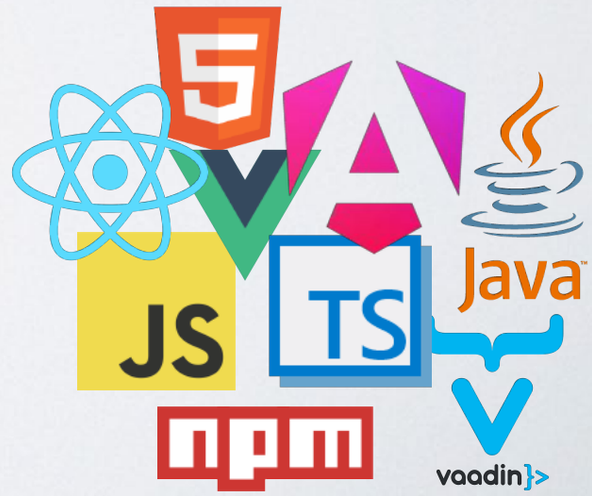
Berner Fachhochschule
Haute école spécialisée bernoise
Bern University of Applied Sciences

letsboot.ch
swiss dev training

digicomp

JavaScript / Angular / React / Vue / Vaadin
Schulung / Beratung / Coaching / Reviews

jonas.bandi@ivorycode.com



I am a fullstack developer with a focus
on modern frontend development.

My expertise is mostly in traditional enterprise systems
(Java, .NET) with modern frontends (React, Angular)

I work on projects, where the end-user
interacts with a browser.

The browser-based user interface is an
abstraction on top of the business functionality.

This talk explores a path into the future where
the last two points are no longer the de-facto
standard ...

The Story:

We used to add chatbots to our apps.
Now we add our apps to the chatbots.

- Kent C. Dodds

https://www.epicai.pro/chat-gpt-app-code-walkthrough-kbh l e

AGENDA

What is MCP?

Chapter 1: Building the AI into our App

Chapter 2: Building our App into the AI

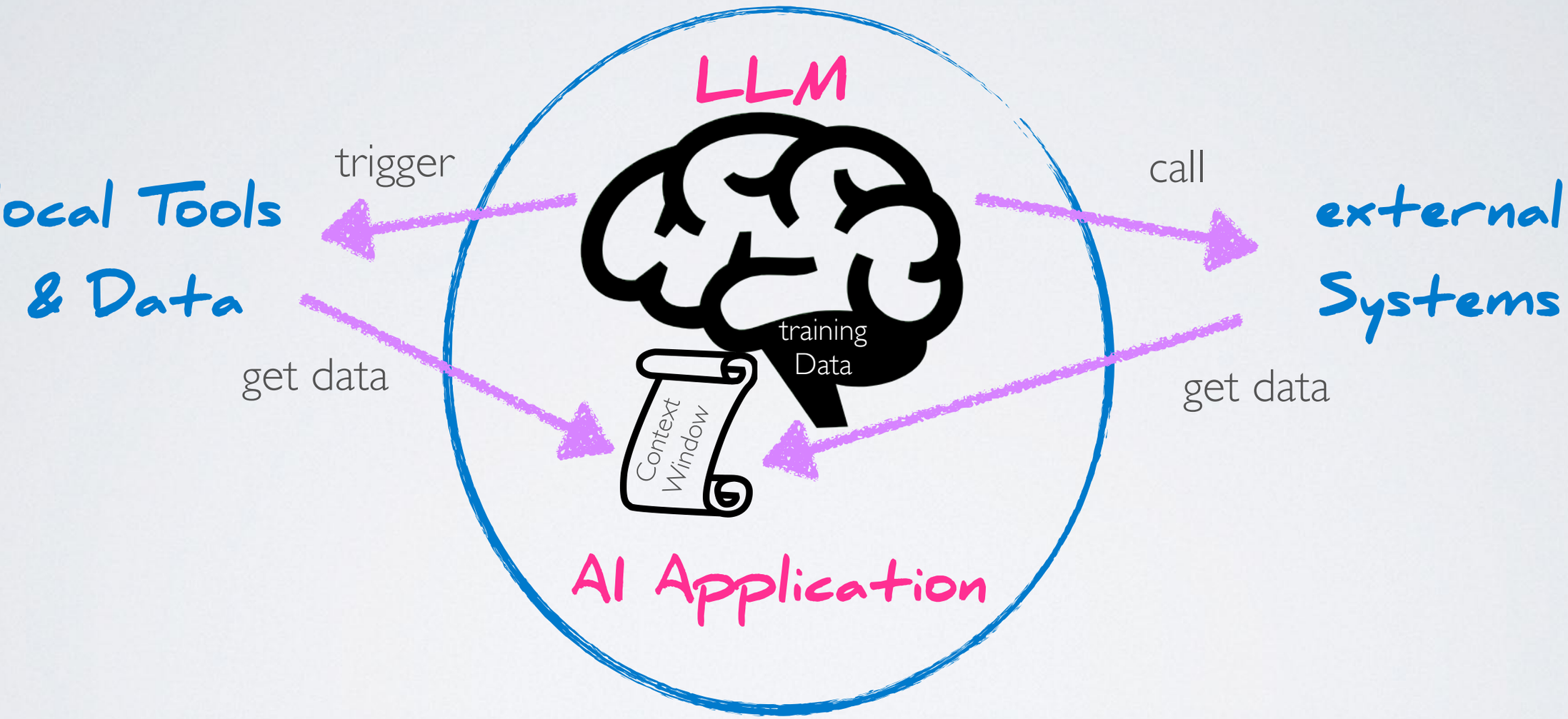
Chapter 3: Something completely different: WebMCP

How does it work: It's all just Tool-Calling

Slides & Code:
https://github.com/ivorycode/mcp-baernerjs-2026/

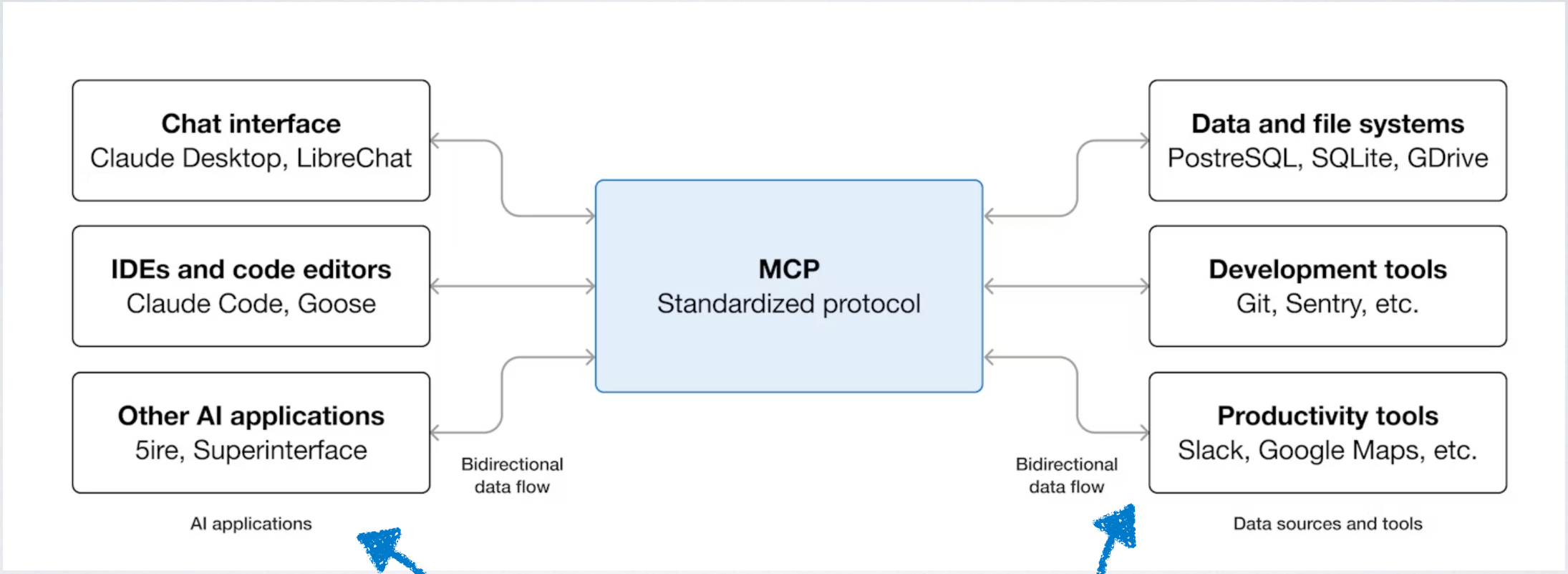
What is MCP?

MCP (Model Context Protocol) is an open-source standard for connecting AI applications to external systems.



MCP standardizes how an LLM interacts with the outside world.

What is MCP?



Somewhere here is an LLM



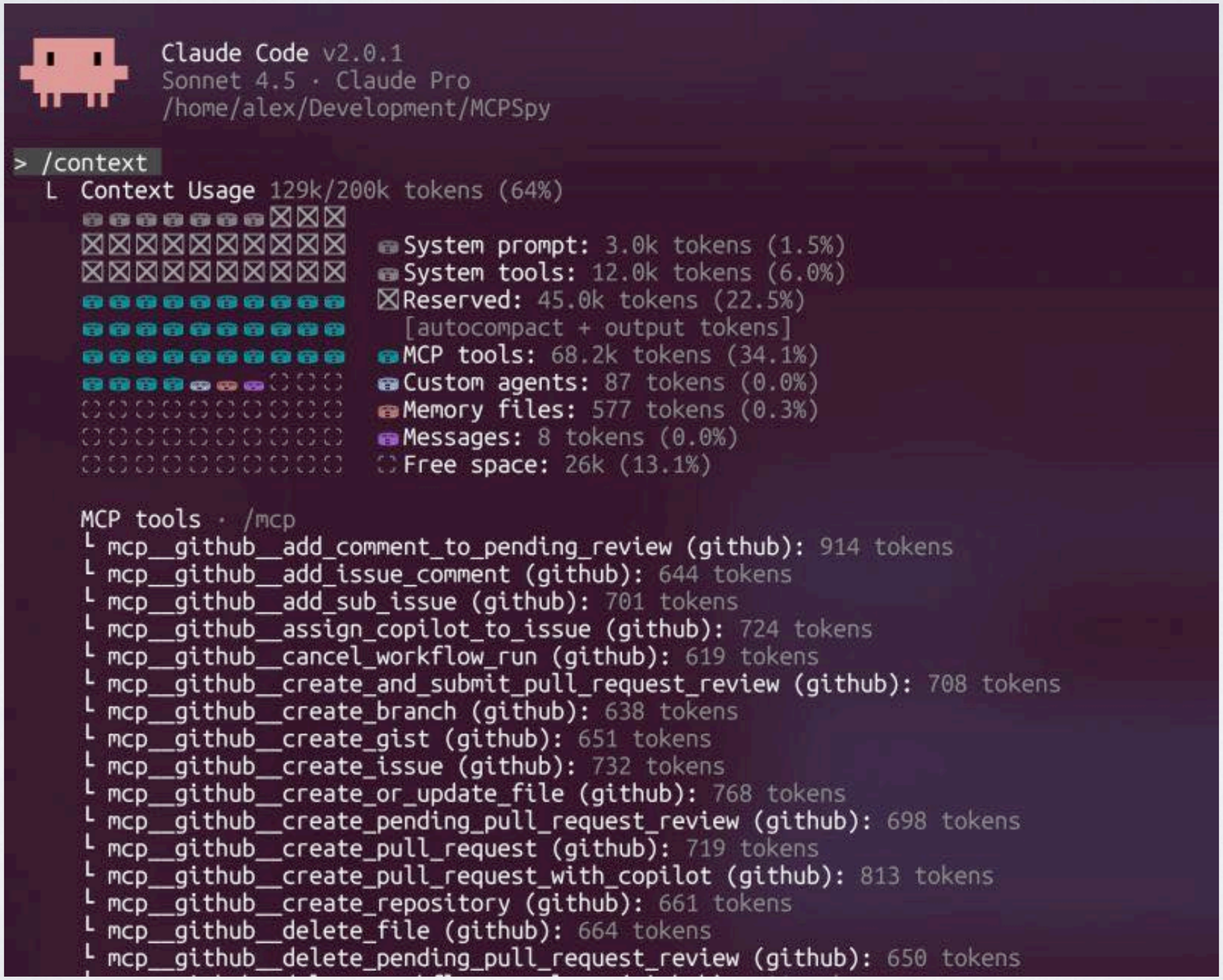


tools exposed by local or remote MCP Servers

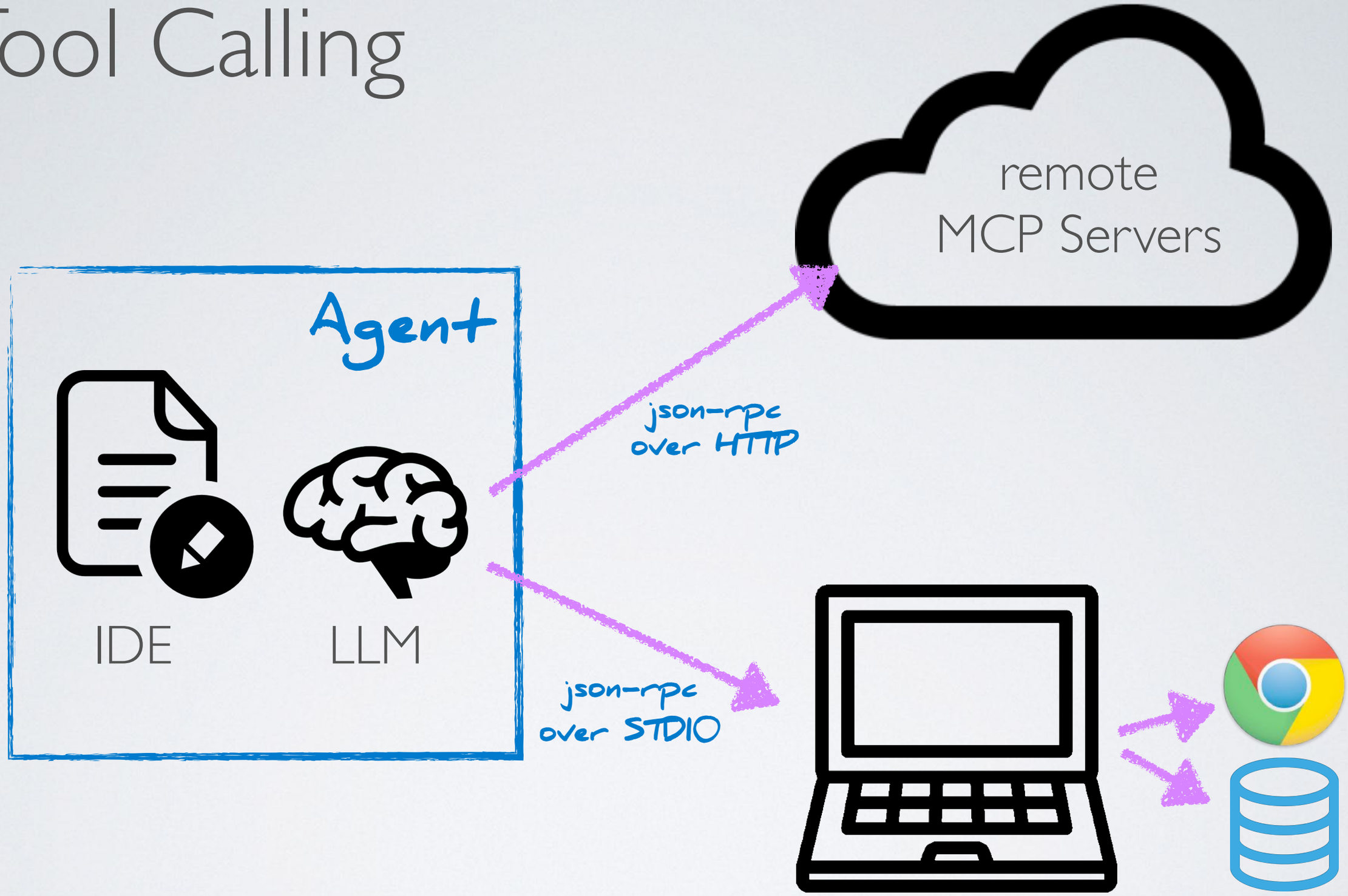
https://modelcontextprotocol.io/docs/getting-started/intro

Who is already using MCP?

(Model Context Protocol Servers)



Tool Calling



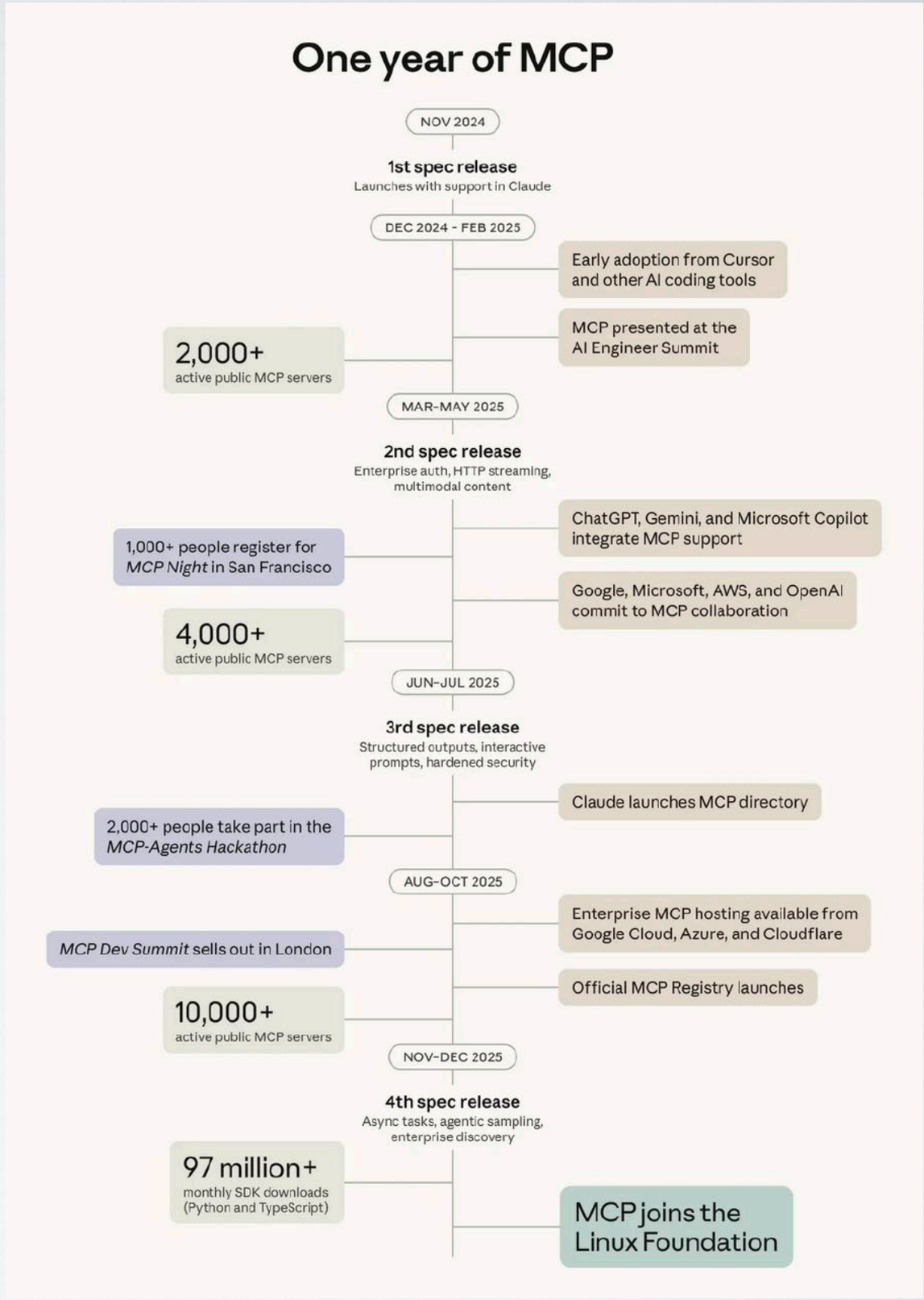
https://context7.com/docs/resources/all-clients
https://github.com/microsoft/playwright-mcp
https://www.atlassian.com/platform/remote-mcp-server
https://www.oracle.com/mcp/

MCP is a standard/protocol initially created by Anthropic in November 2024

MCP is now part of the Linux Foundation

- In December 2025 the Agentic AI Foundation (AAIF) was founded
- MCP is now developed as an open standard
- 2026-07-28: major spec update

WebMCP is a proposed web standard mainly driven by Google and Microsoft.
It is not part of the MCP standard.



https://www.anthropic.com/news/donating-the-model-context-protocol-and-establishing-of-the-agentic-ai-foundation



MCP vs. CLI + Skills

The trend in agentic engineering :
The MCP-Hype is over, the trend goes to CLI + Skills

Examples:

- https://github.com/microsoft/playwright-cli
- https://github.com/upstash/context7?tab=readme-ov-file#cli-commands

CLI trade-offs:
+ token efficiency & context efficiency / keep data out of context
+ "simple technology" (i.e. auth outside of coding agent)
- must be installed locally

stdio MCP-Servers: replacable with "CLI + Skills"
streamable http MCP-Servers: still valuable as centrally deployed and managed components

CURSOR

CLAUDE CODE

JetBrains AI

opencode

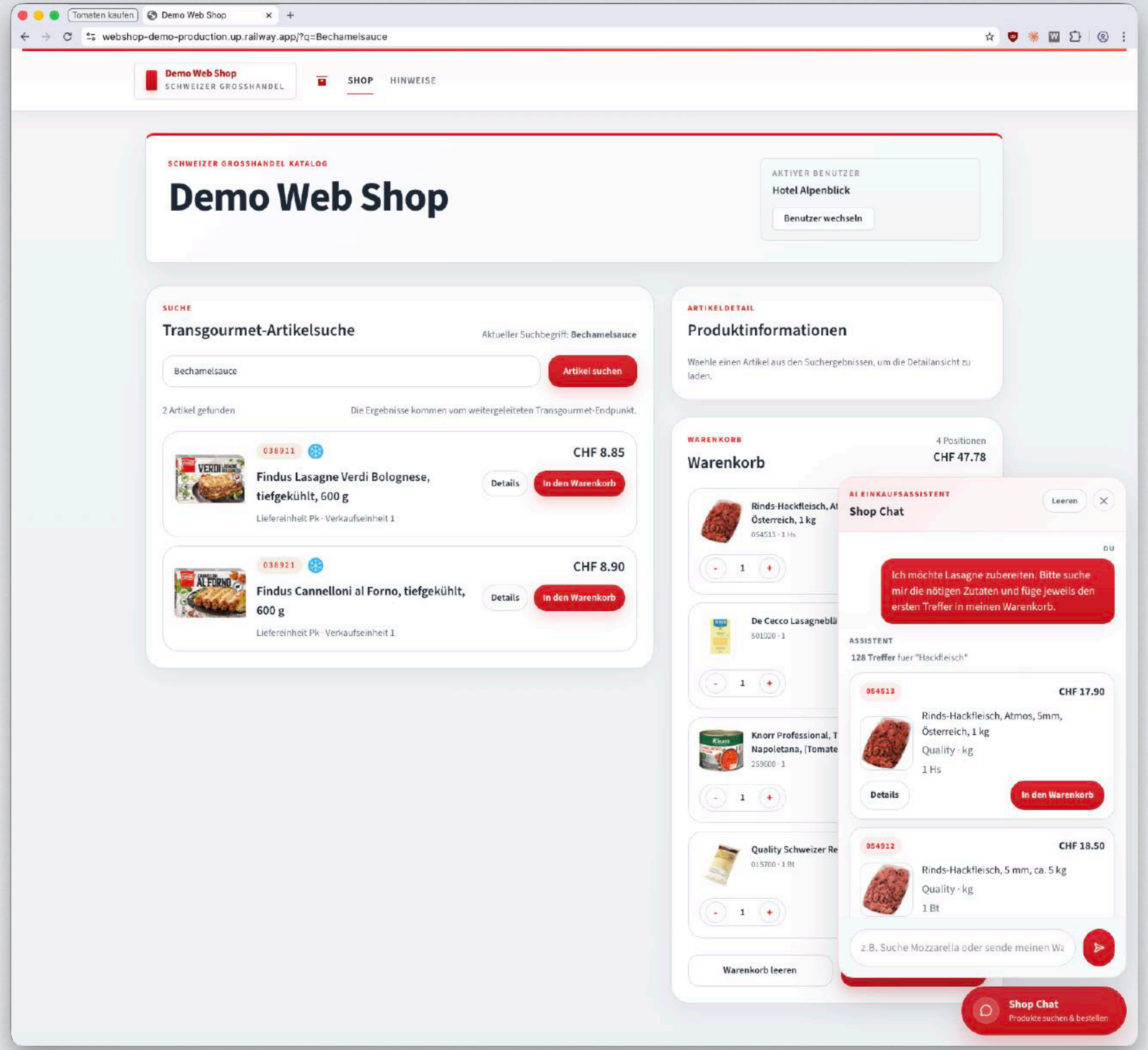


ChatGPT

Claude

A look beyond agentic engineering ...

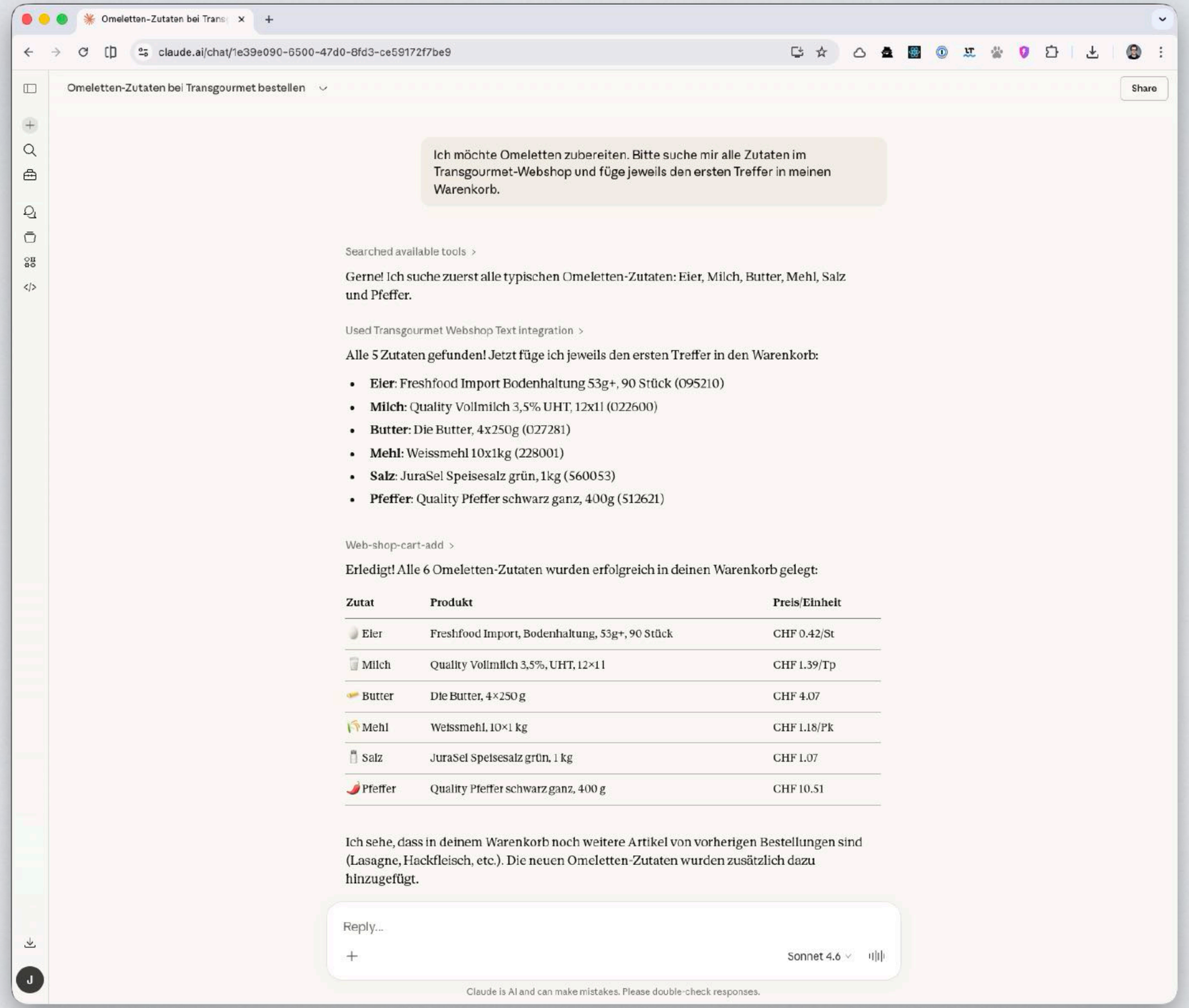
Chapter 1: Building the AI into our Application



Thesis:
AI-Agents are the future of user interactions.

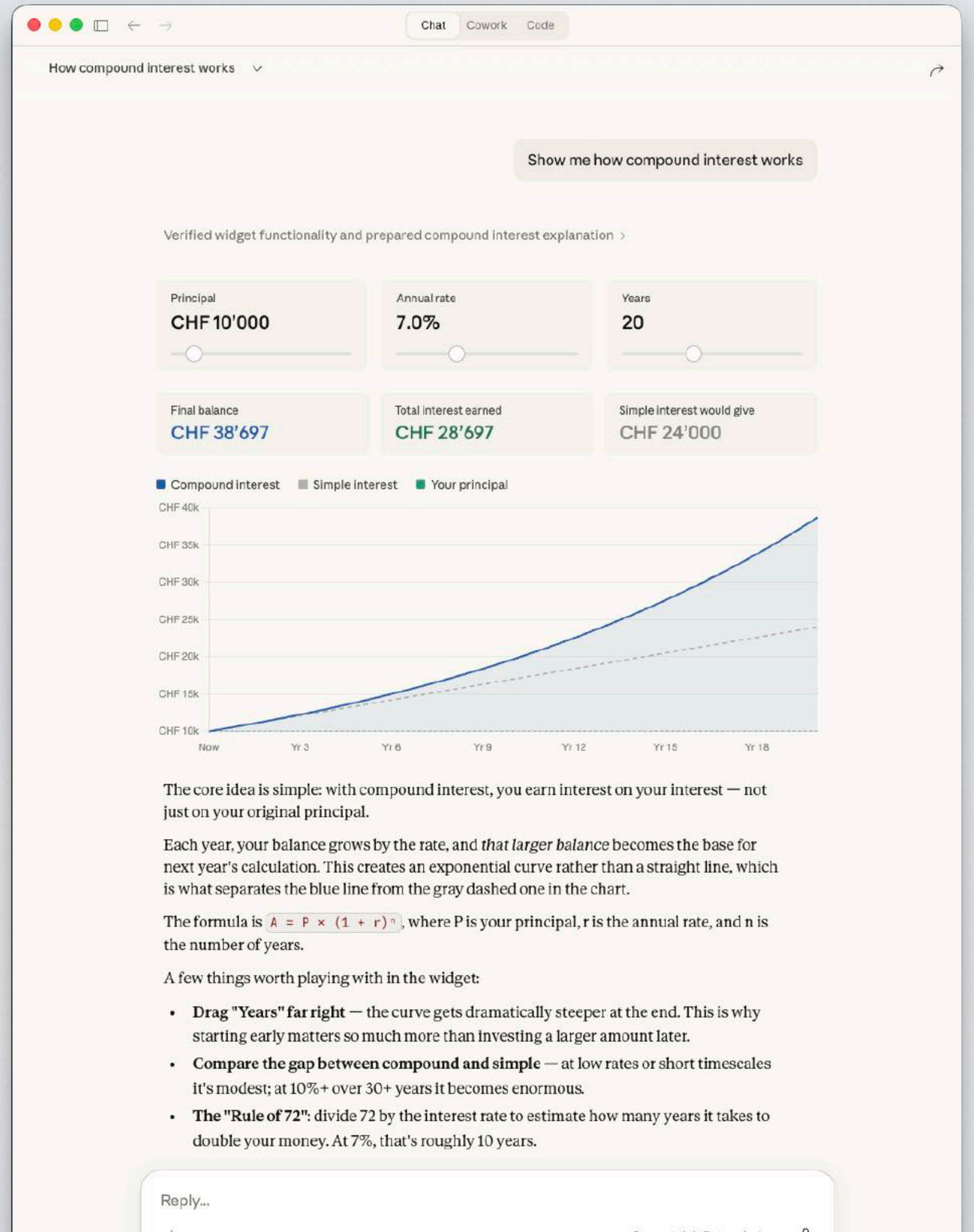


Chapter 2: Building our App into the AI

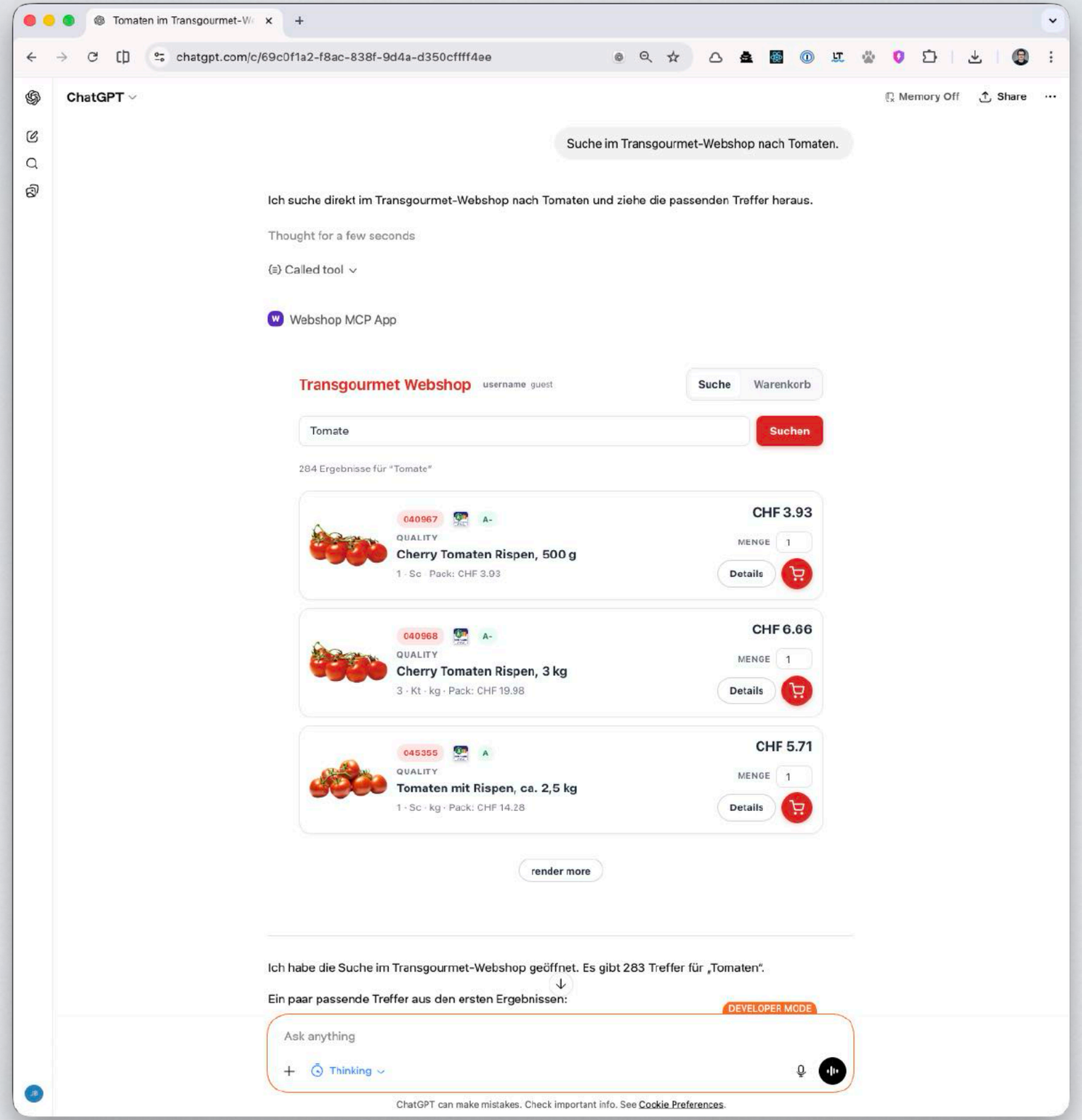


AI Agents: More than Chat

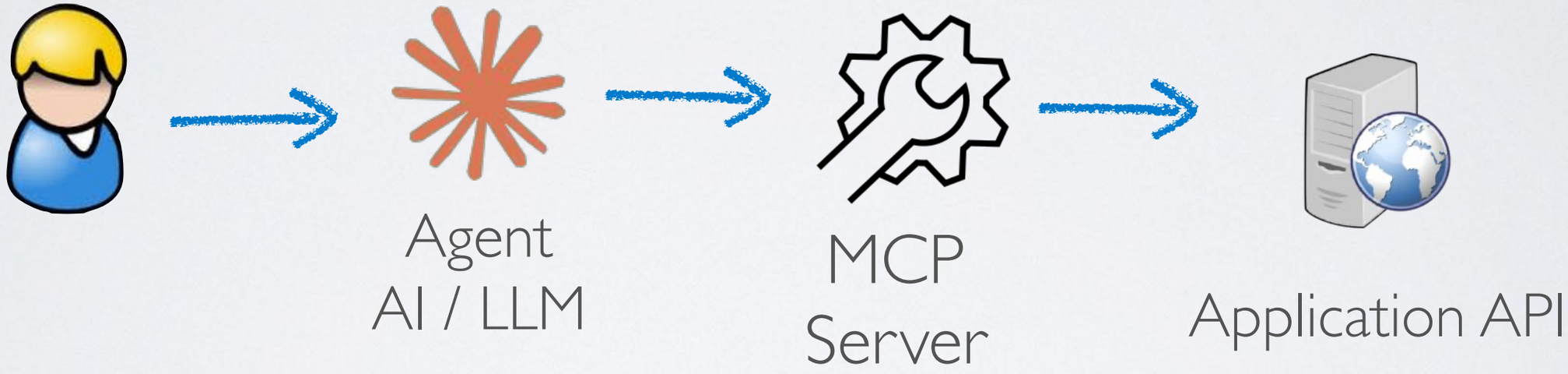
Show me how compound interests work.



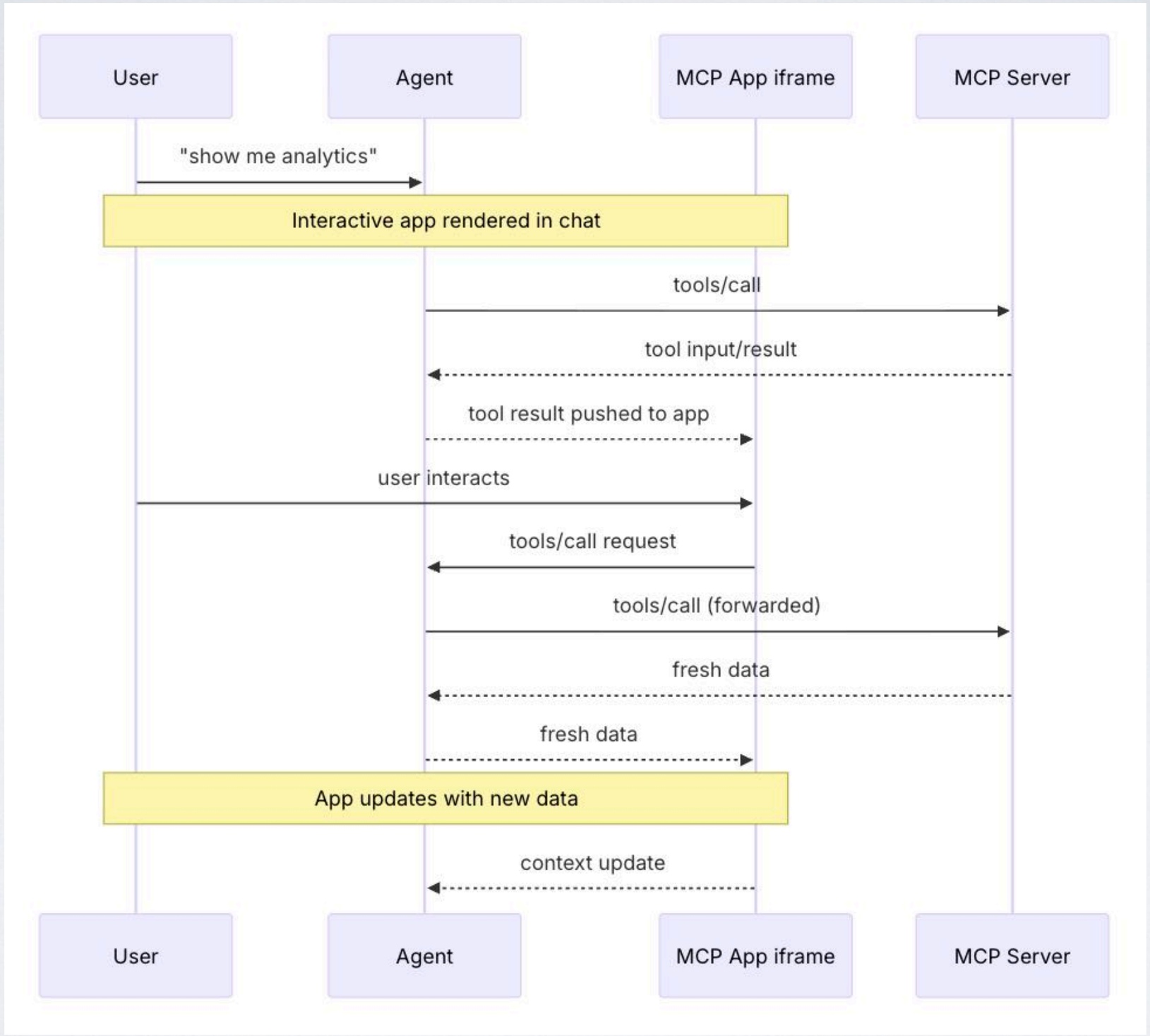
MCP Apps - More than Chat!



Accessing an Application



MCP Apps



https://modelcontextprotocol.io/extensions/apps/overview
https://apps.extensions.modelcontextprotocol.io/api/



MCP Apps

Interactive UI applications that render inside MCP hosts.

Official MCP Extension since January 2026

https://modelcontextprotocol.io/extensions/apps/overview

MCP App evolved out of:

- MCP-UI: https://mcpui.dev/
  (announced in May 2025)
- OpenAI Apps SDK: https://developers.openai.com/apps-sdk
  (announced in October 2025)

MCP-UI is deprecated.

OpenAI App SDK is here to stay.

https://developers.openai.com/apps-sdk/reference
https://developers.openai.com/apps-sdk/mcp-apps-in-chatgpt


AND NOW FOR
SOMETHING
COMPLETELY
DIFFERENT
WebMCP

WebMCP

A proposed Web Standard / Draft Community Group Report

Announced on February 9, 2026

https://webmachinelearning.github.io/webmcp/

https://github.com/webmachinelearning/webmcp

A way for web pages to expose structured tools for AI-agents.
Improving the reliability and performance of interactions between web page and AI-agent.

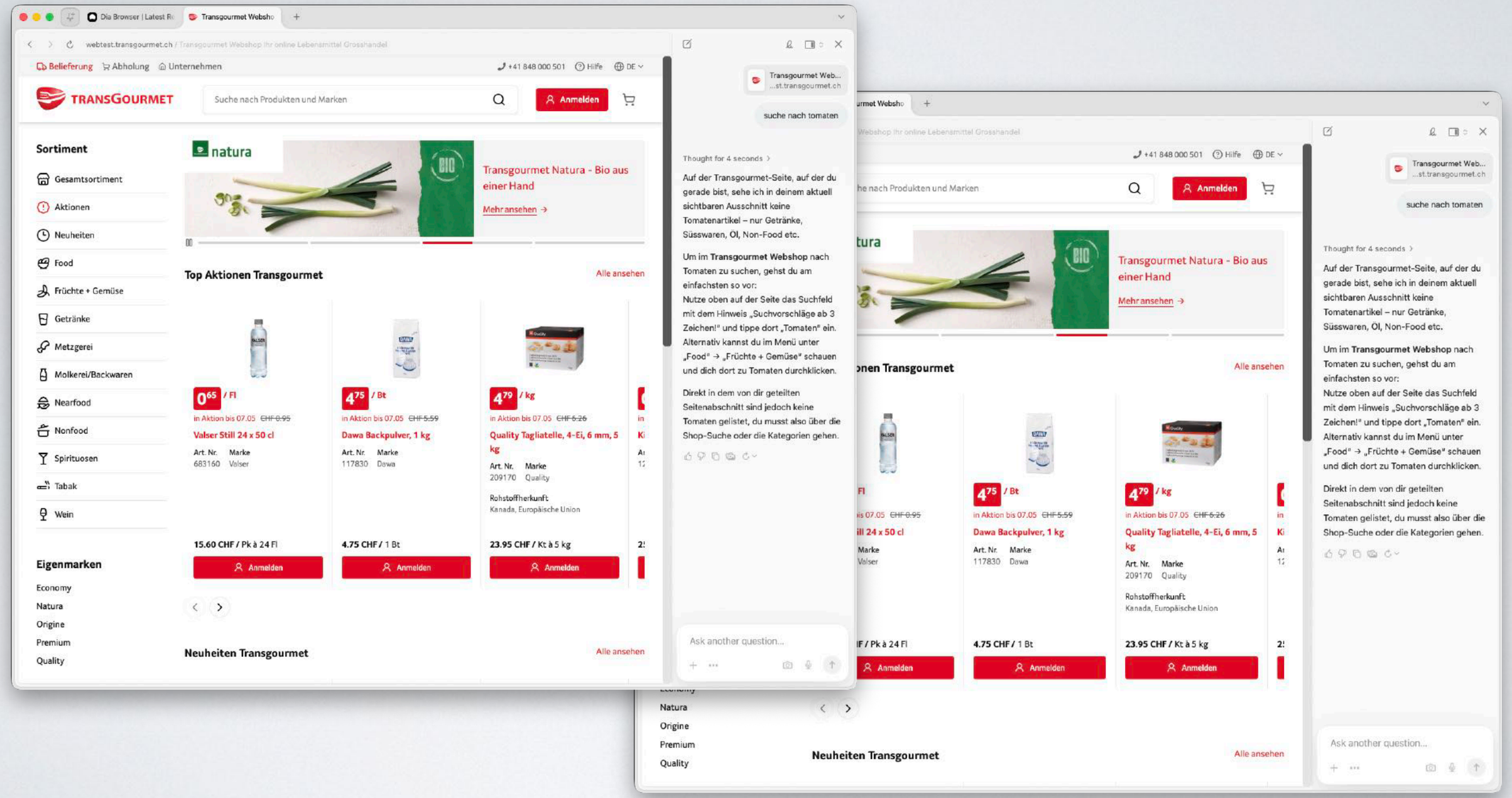
Similarity to MCP:

- It is a mechanism to expose tools to an agent

Differences:

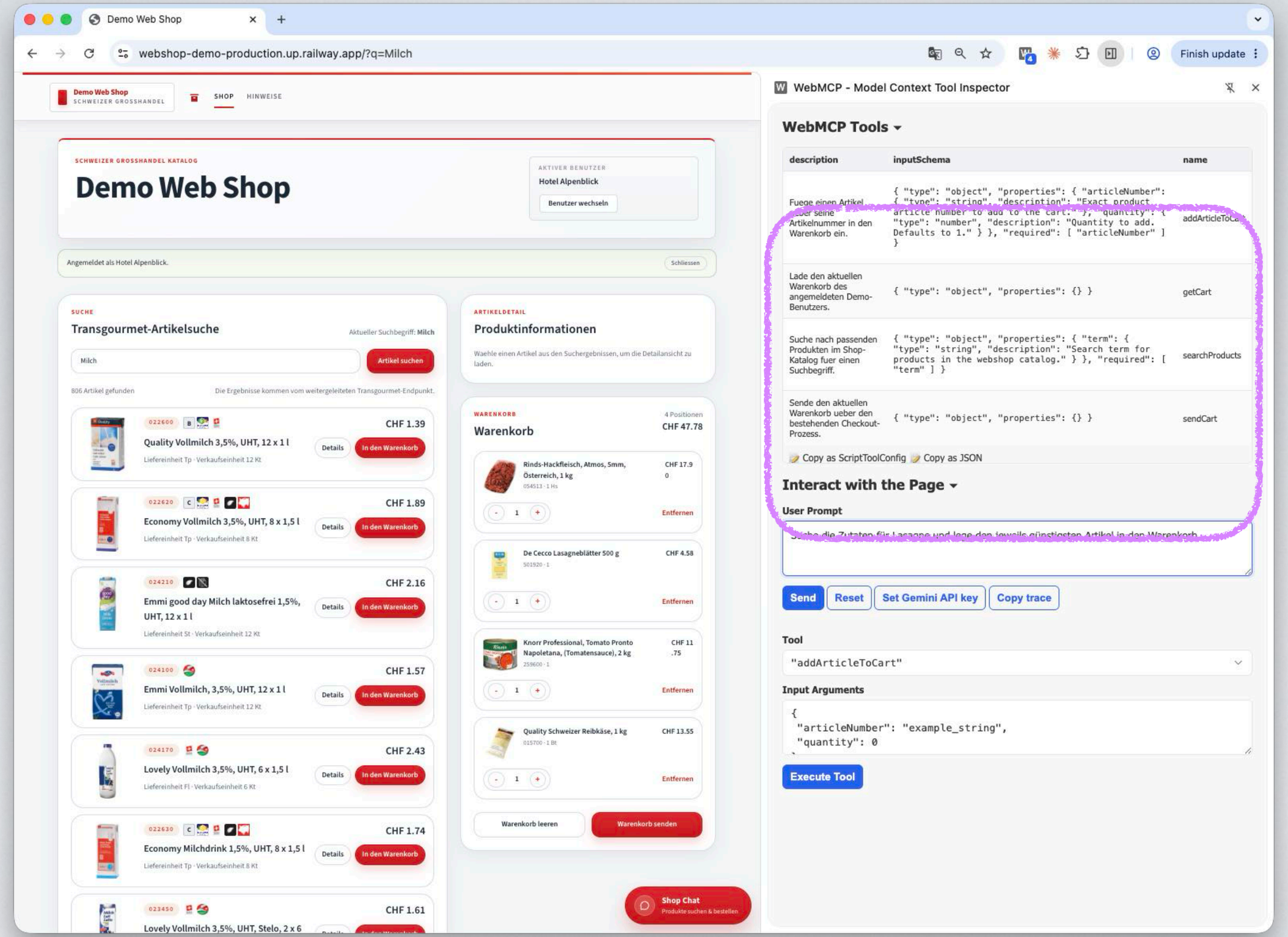
- client-side mechanism
- DOM-API
- no "transport protocol" involved

"Smart" Browsers



Dia, Atlas, Comet ... Claude Browser Extension ...

Chapter 3: Agent controlled App




```
navigator.modelContextTesting.executeTool("searchProducts", JSON.stringify({term: 'Milch'}))
```

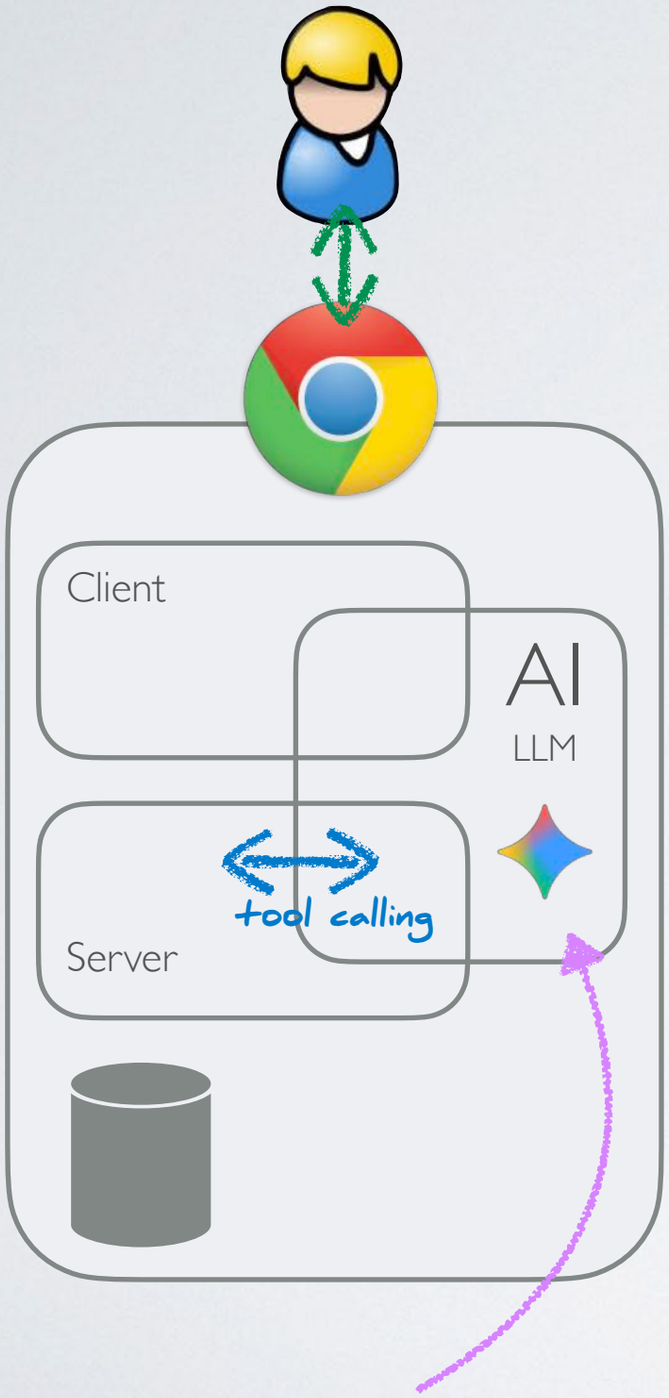
Chrome Extension: https://chromewebstore.google.com/detail/webmcp-model-context-tool/gbpdfapgefenggkahomfgkhfehlcenpd?pli=1

Summary

Three Mechanisms how AI can control our app:

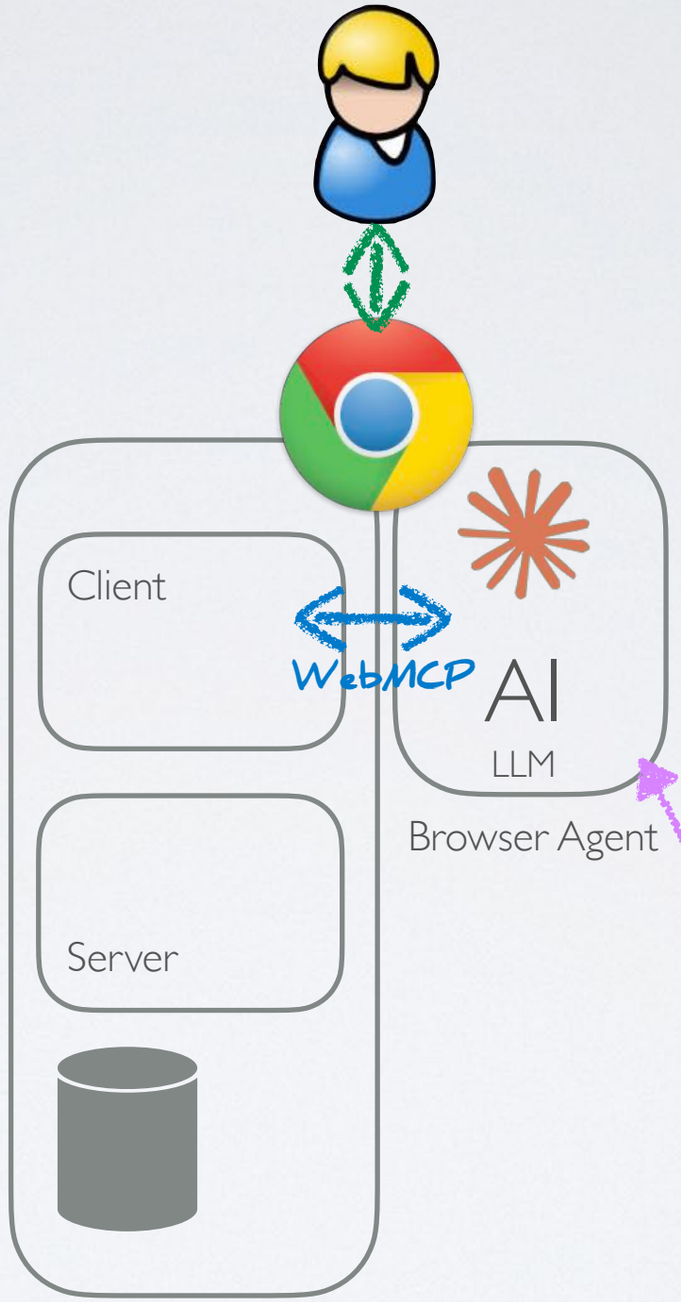
- AI inside the App
- The App in the Agent
- The agent-controlled App

AI in the App

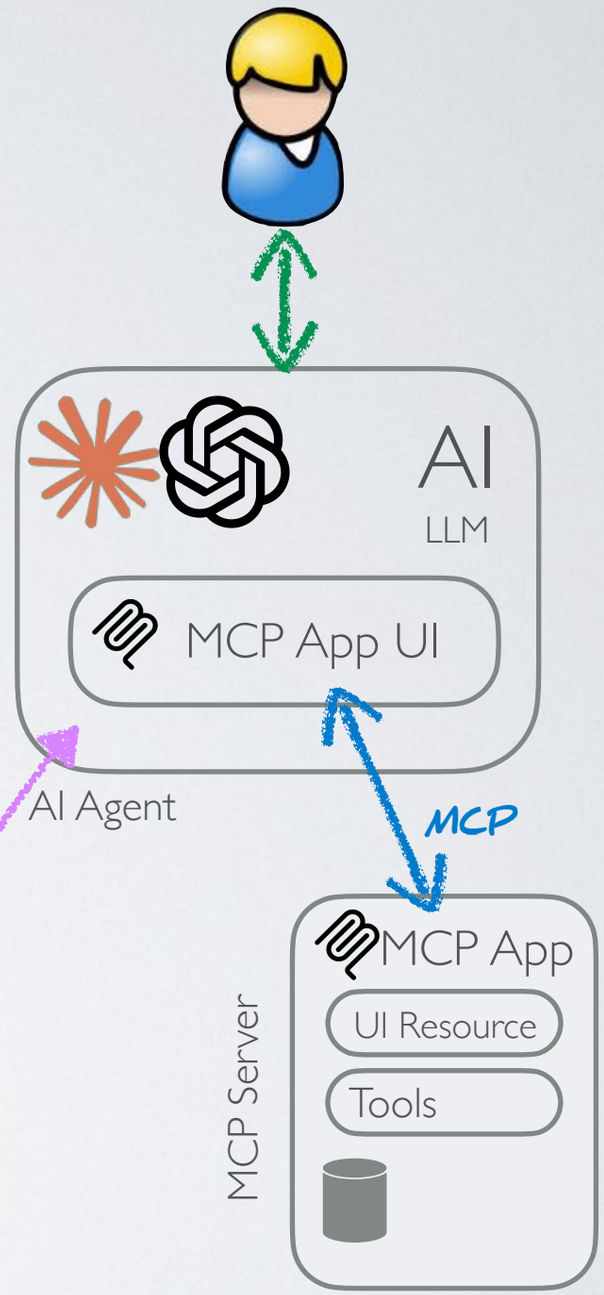


App pays for LLM

AI-controlled App



MCP App



User pays for LLM

New UI Paradigms?

How do you "navigate" in a chat-bot?

"Components will kill Pages ..."
=> Agentic UI, AG-UI, A2UI ...

AI will be the new steering wheel of the internet, it won't be navigating pages.

https://bitsandbytes.dev/posts/components-will-kill-pages

Is it ready for production?

Like with most things AI, there is a big hype around MCP.

There are some shiny demos.

For me it feels like the early browser days:
features are very basic
duck-tape implementations

But it might worth to keep an eye on:
frameworks and patterns might evolve very quickly.

Personally I don't think it will replace browsers anytime soon.
(And I think I would not like a future where it does replace browsers)

Questions?

How does it Work?

It's all just "tool-calling"

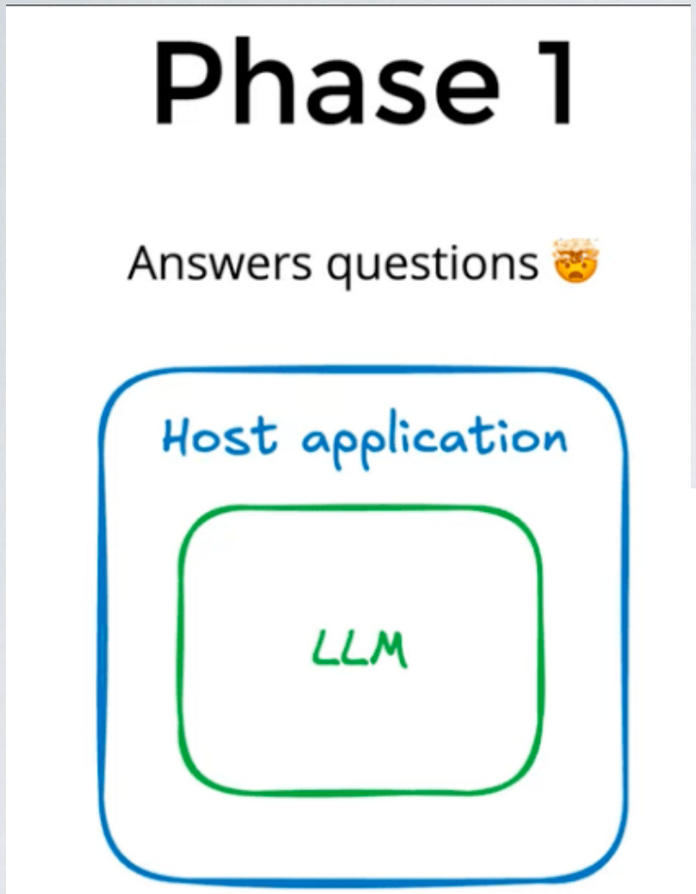
Many modern LLMs are trained to use tools.
It's one of the pillars of Agentic AI.

"Deployment"

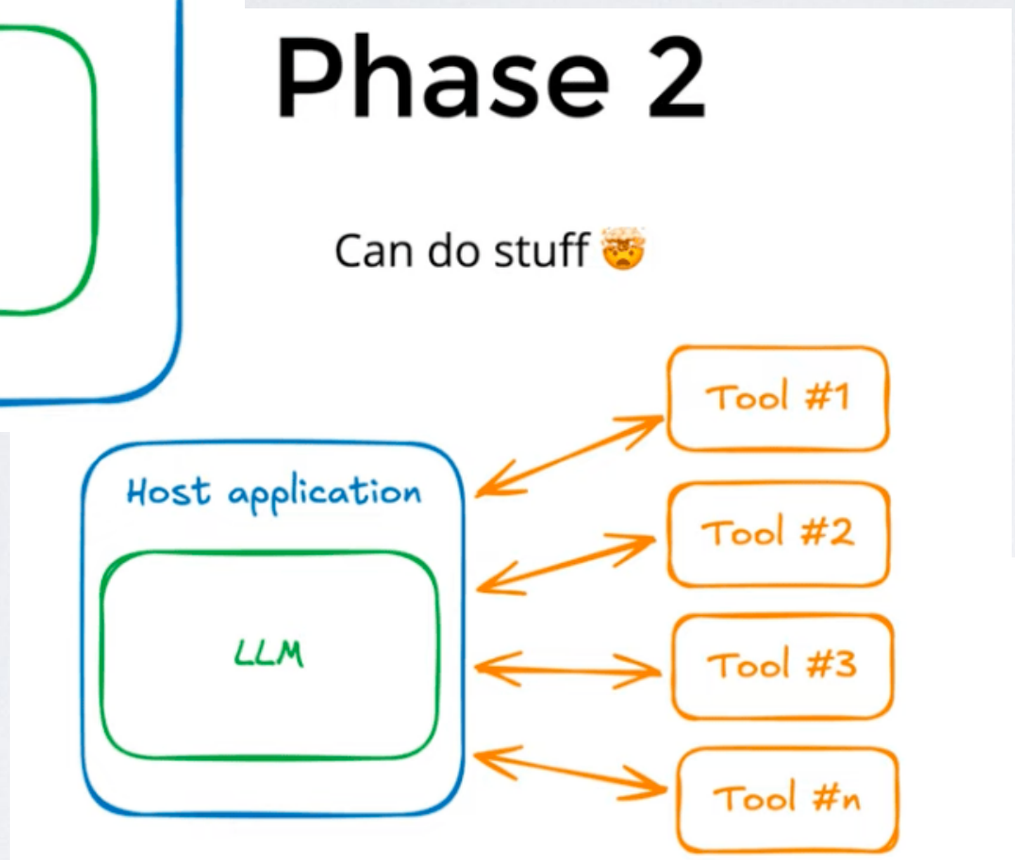
- Providers can "deploy" their MCP Apps similar to Websites
  - Discovery currently happens through an official "app-store like" channel.
  - Dedicated Hosting Platforms i.e. https://alpic.ai/
- End-User "installs" the App in his Client (Claude -> *Connectors*, ChatGPT -> *Apps*)
  - similar to installing a "Mobile App" from the app store ... not like loading a webpage
  - Proprietary "App Stores" (i.e. Claude Connectors, ChatGPT Apps ...)
  - Future: maybe automatic "discovery" of some "featured" apps?
  - Future: maybe public registry with "discovery"?

Anthropic MCP Directory Submission Form: https://docs.google.com/forms/d/e/1FAIpQLSeafJF2NDI7oYx1r8o0ycivCSVLNq92Mpc1FPxMKSw1CzDkqA/viewform
ChatGPT Apps Directory Submission: https://developers.openai.com/apps-sdk/deploy/submission

History of Tool Calling

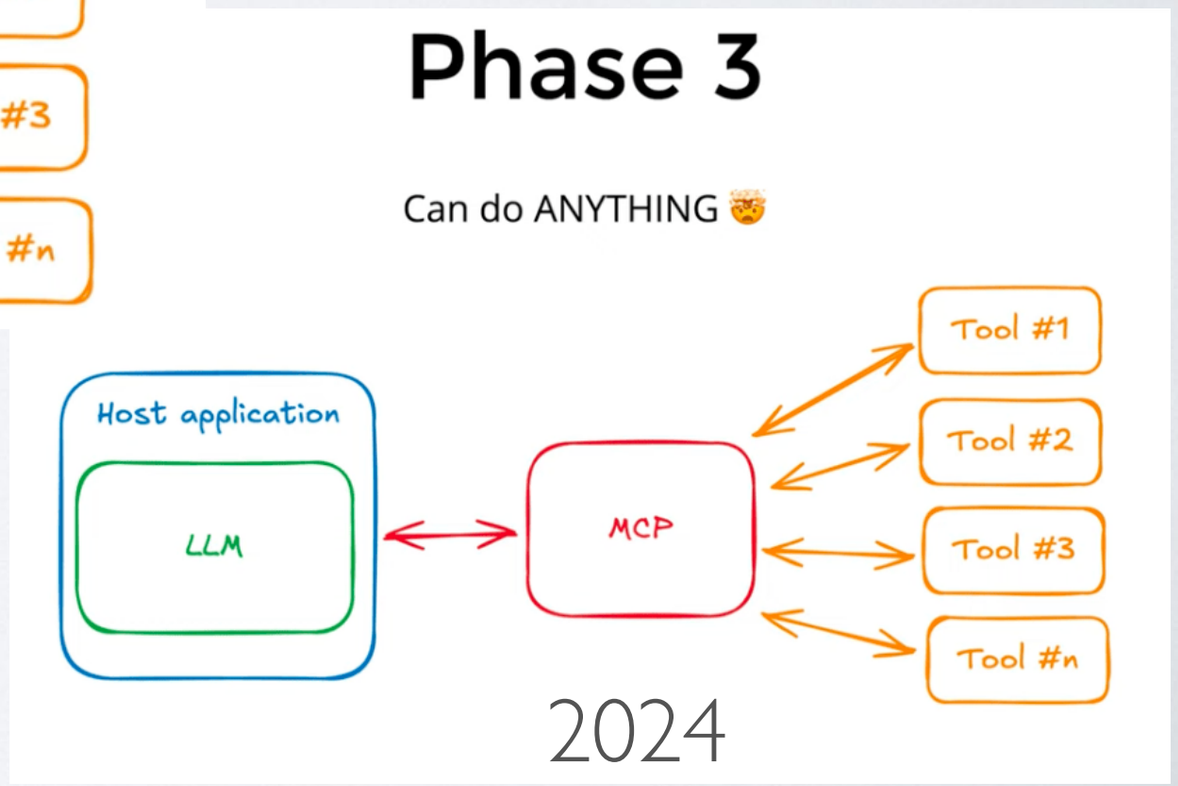


2022



2023

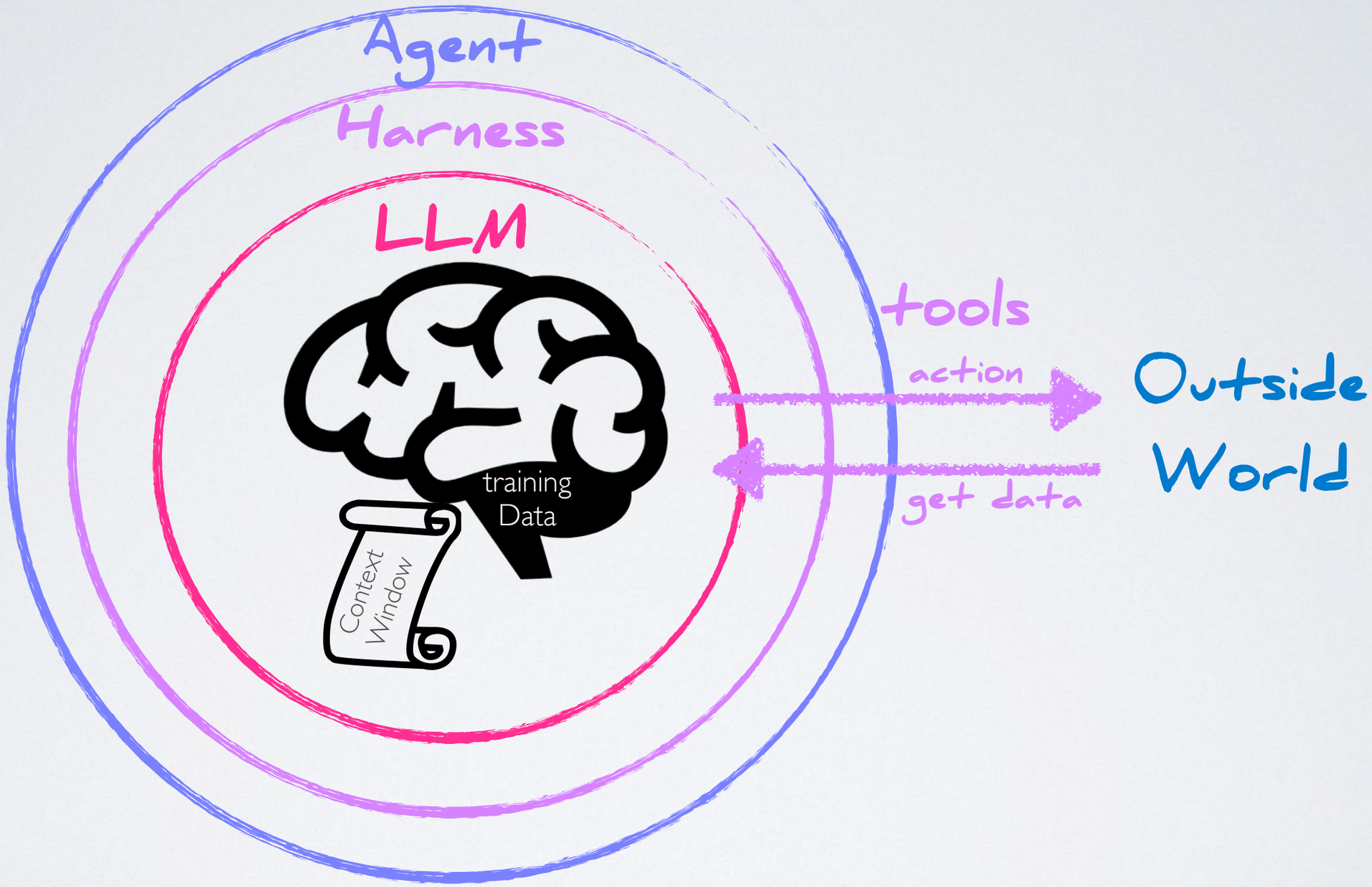
MCP is the "USB port for AI"



2024

https://www.youtube.com/watch?v=EyZiAp0pelw

What is an agent anyways?

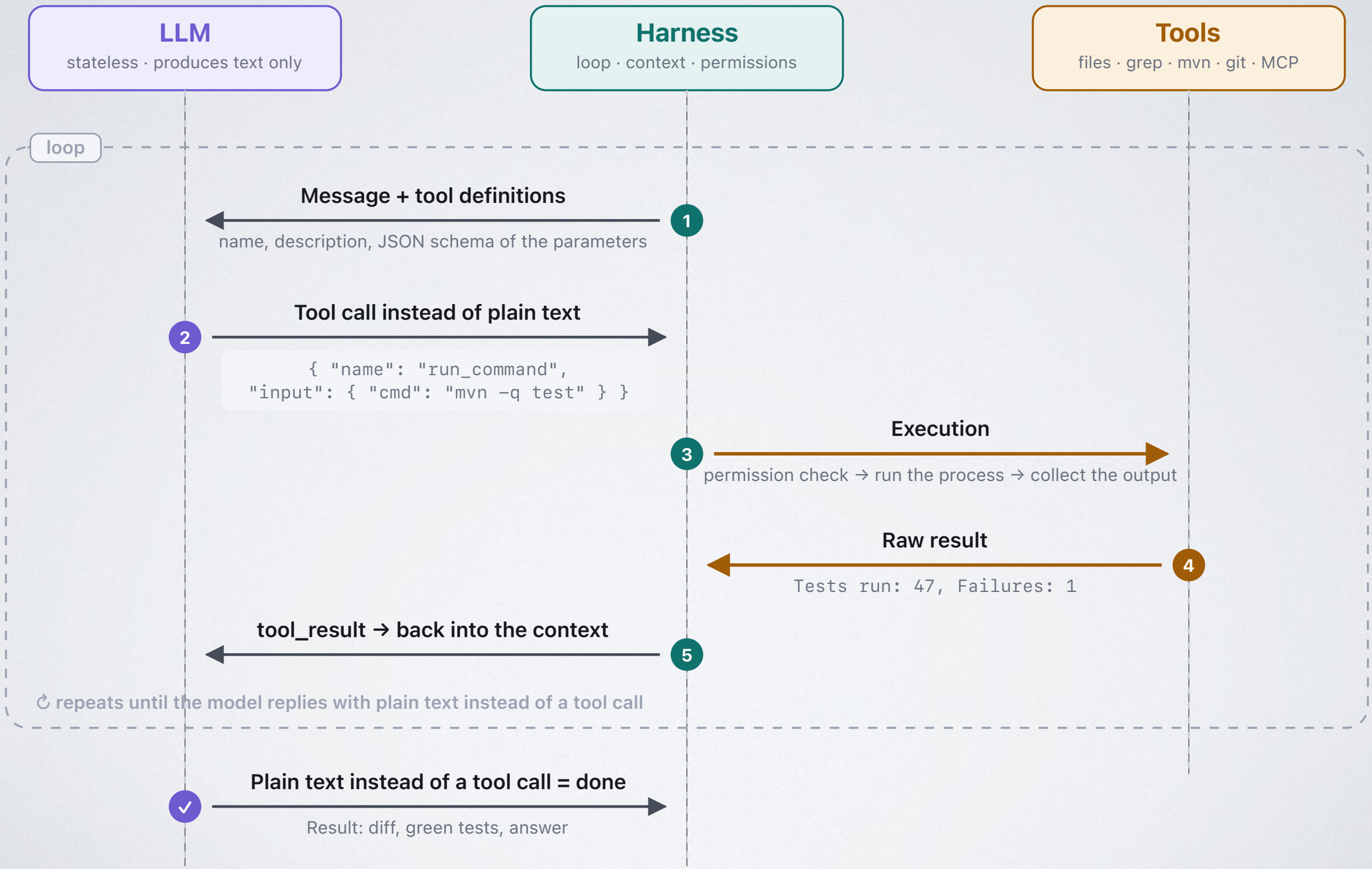


LLM + Harness = Agent

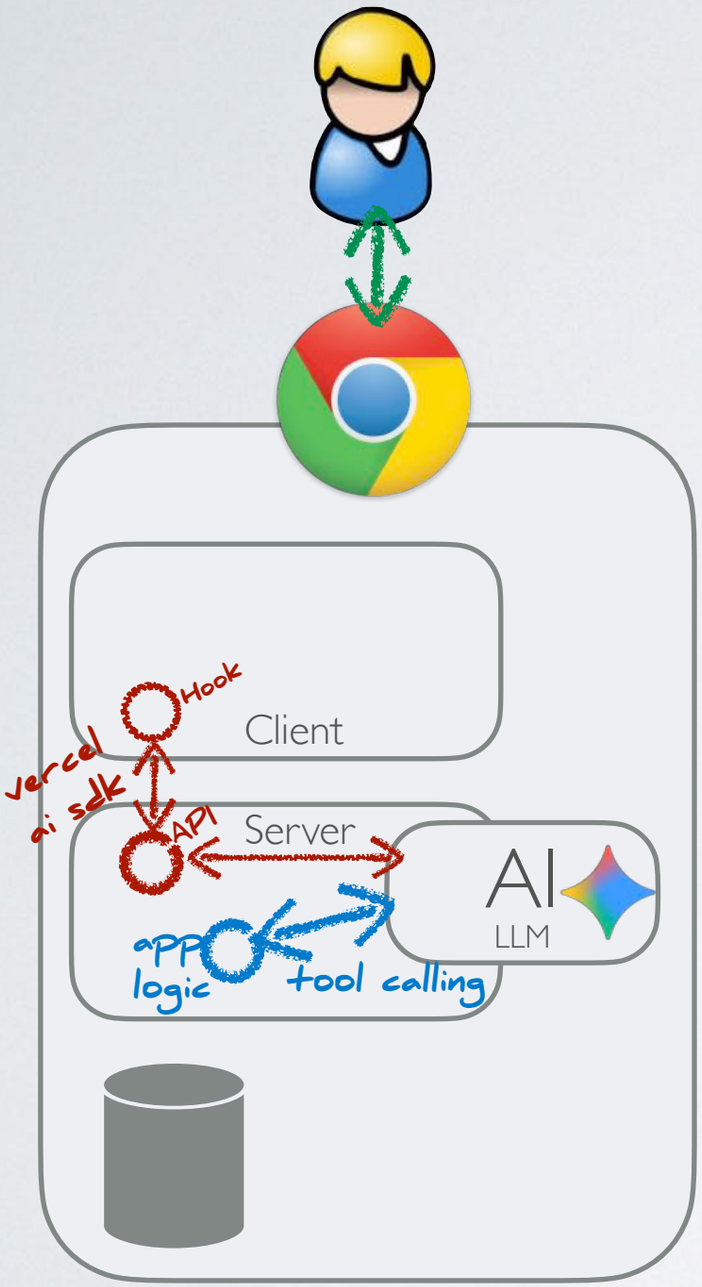
Harness: the programm around the LLM: looping & orchestration, tools, permissions, context management ...

Agent: lives in a context - like your local computer (codex, cli), in the ChatGPT sandbox ...

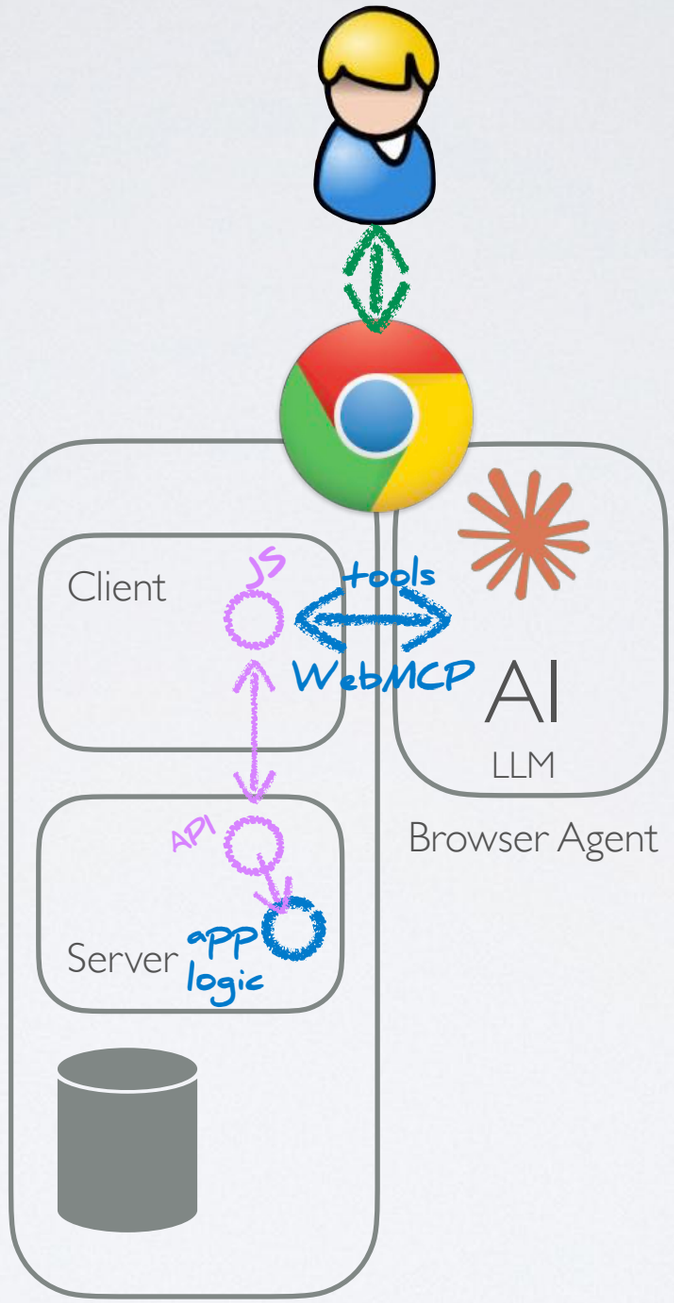
Tool Calling: From LLM to Agent



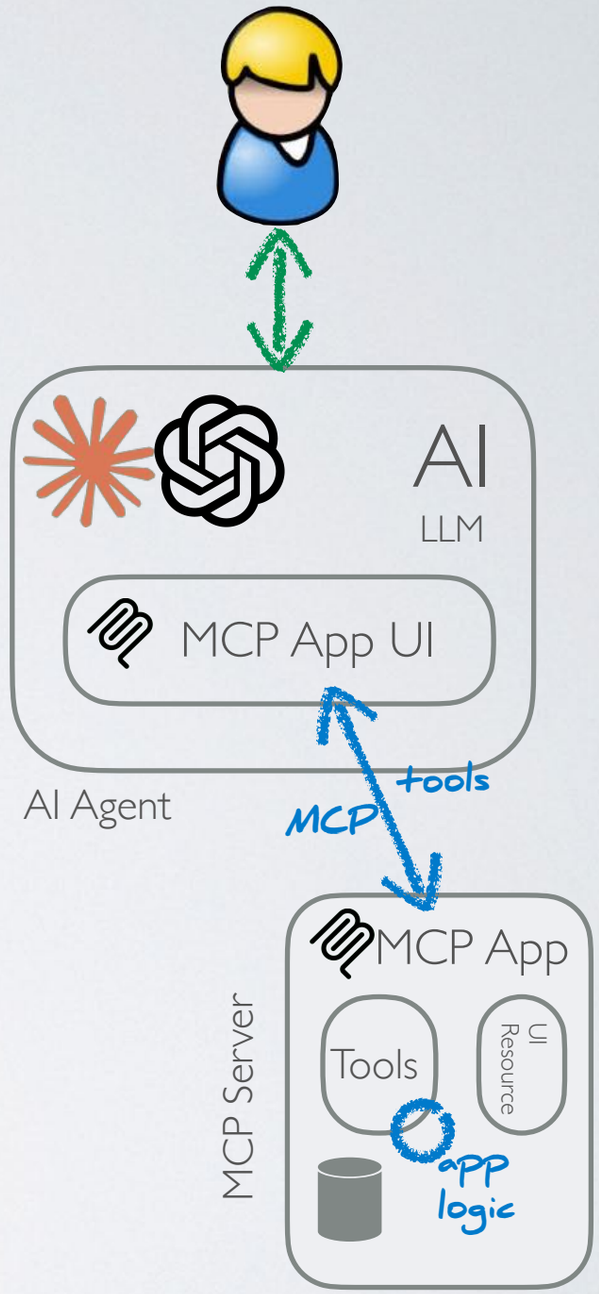
AI in the App



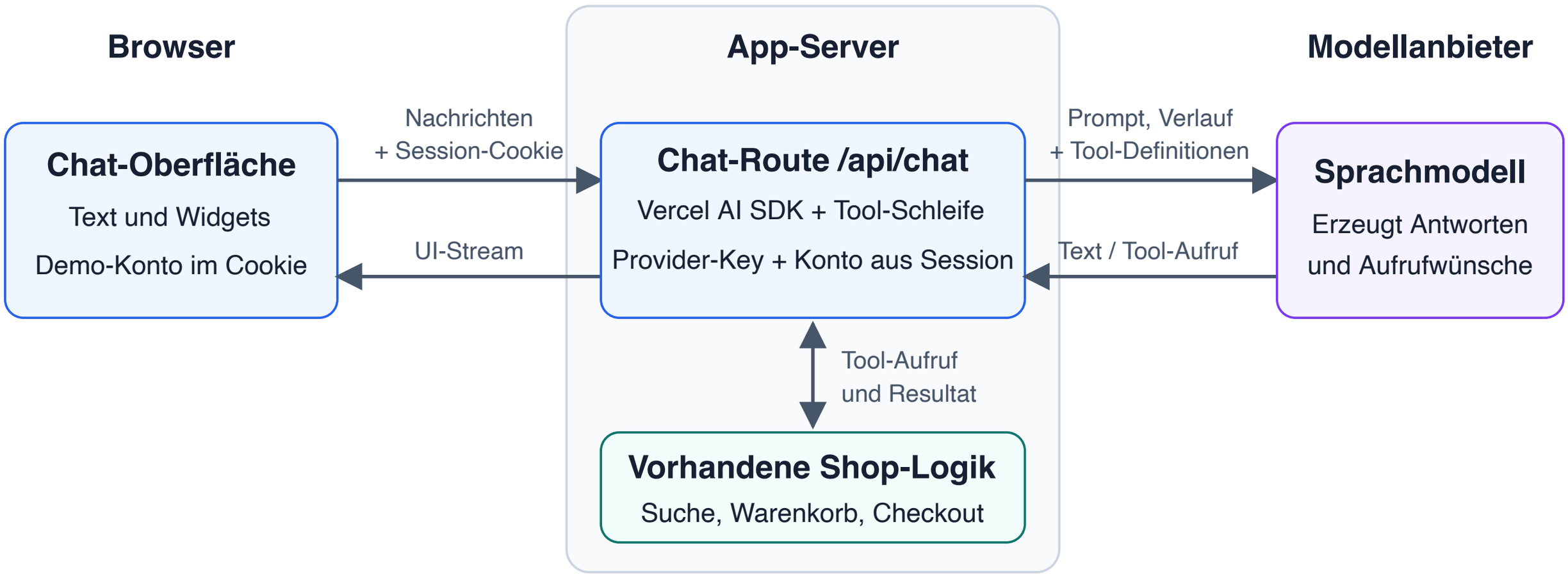
AI-controlled App



MCP App

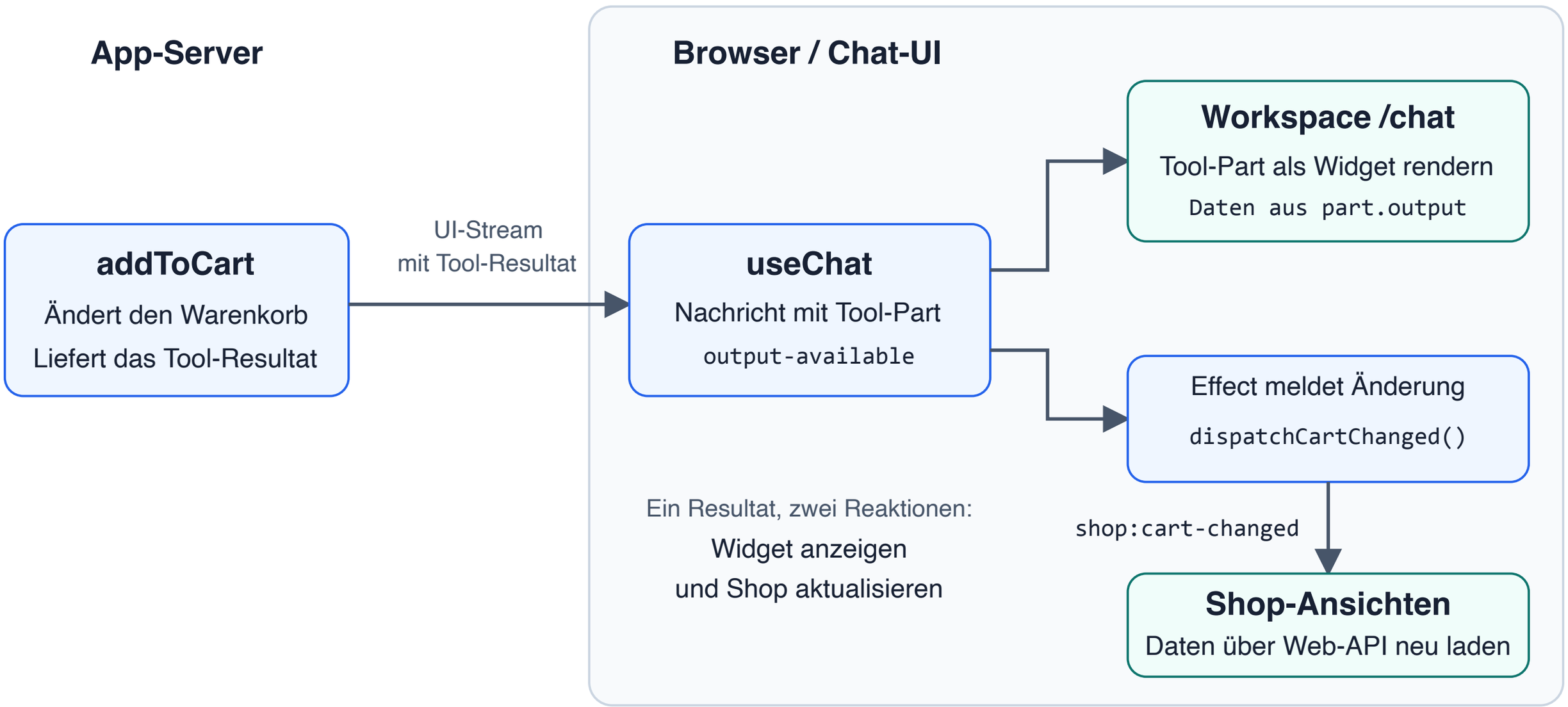


Architektur: AI in App



Der API-Key und die Tool-Ausführung bleiben auf dem App-Server.

Implementation: AI in App

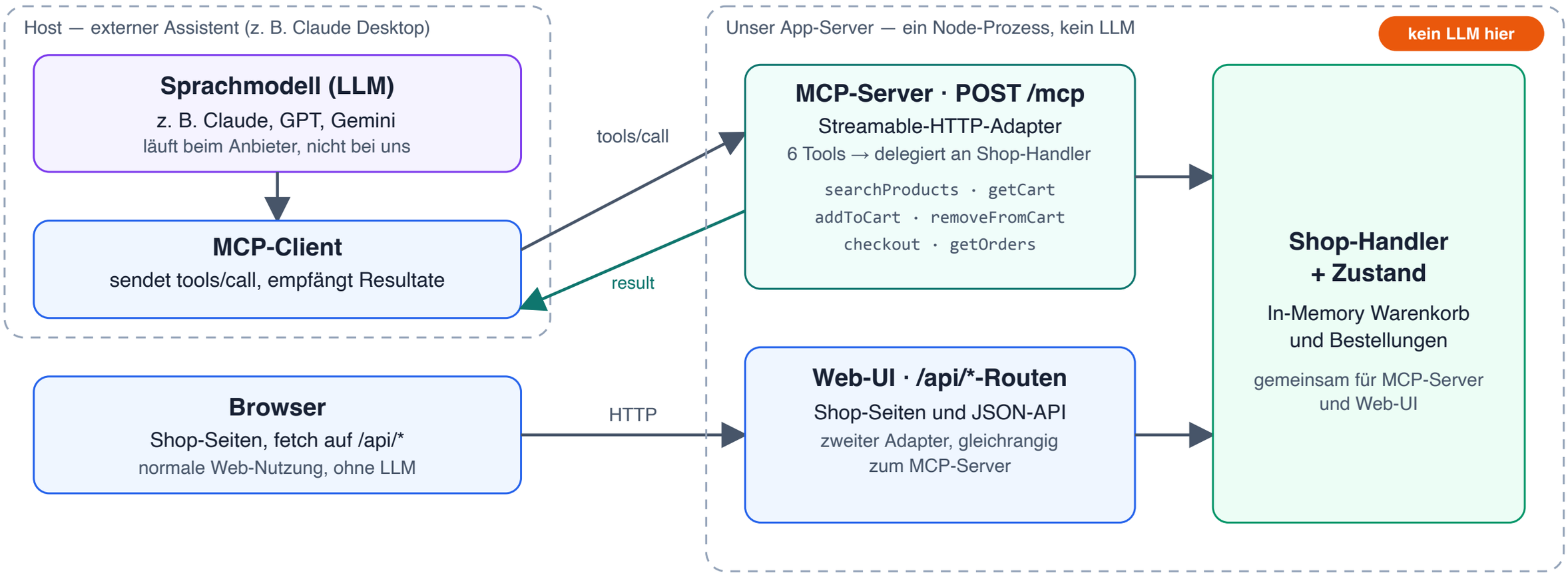


Das Shop-Event meldet die Änderung. Die aktuellen Daten kommen aus der Web-API.

Architektur: App in AI

Der Webshop als MCP-Server

Der externe Assistent bringt Modell und MCP-Client mit — unser Server führt nur die Tools aus



Merke: Das Sprachmodell steckt im Host des externen Assistenten, nie in unserem MCP-Server.
Unser Server bleibt reine Tool-Ausführung — Web-UI und MCP-Server teilen sich im selben Prozess denselben Warenkorb.

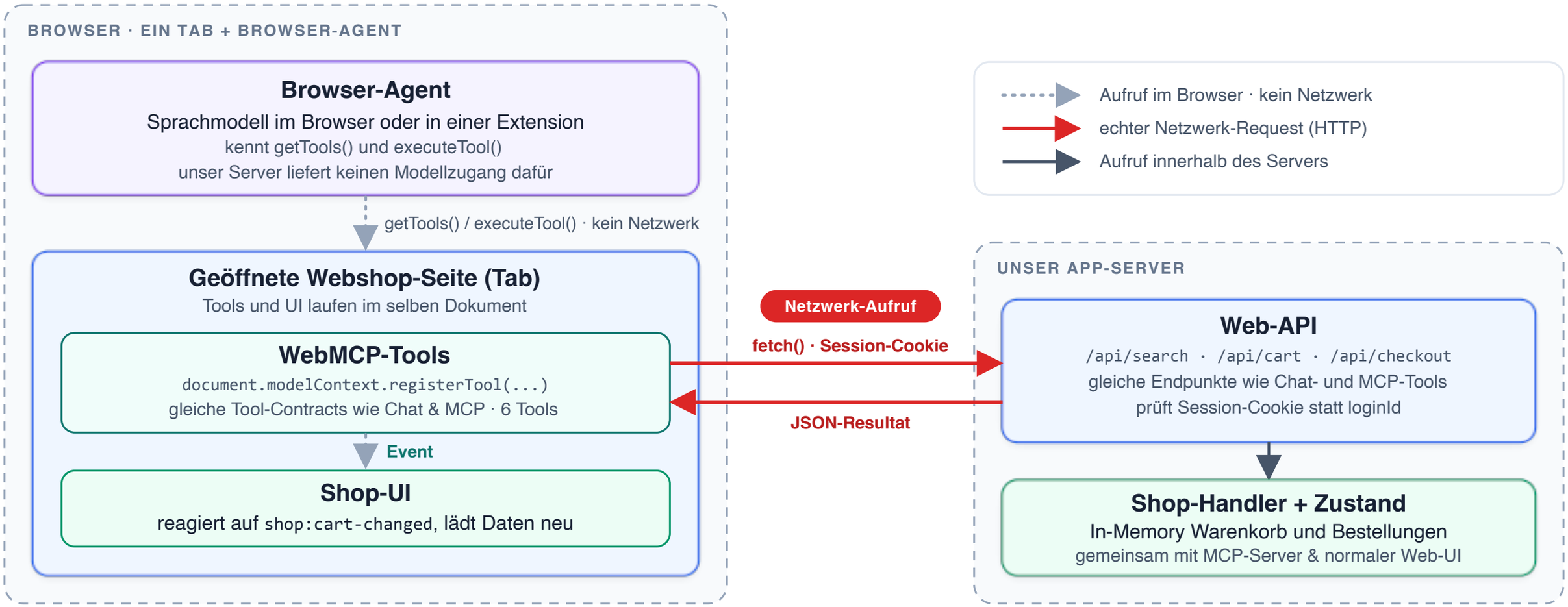
MCP Server


MCP-Server
unsere Anwendung
stellt Tools, Resources und Prompts bereit
Tools
Fokus im Workshop
Aufrufbare Funktionen
z. B. add(a, b) · searchProducts(...)
Resources
Abrufbare Inhalte
z. B. hello://about
Prompts
Wiederverwendbare Nachrichtenvorlagen
z. B. greet

Architektur: WebMCP

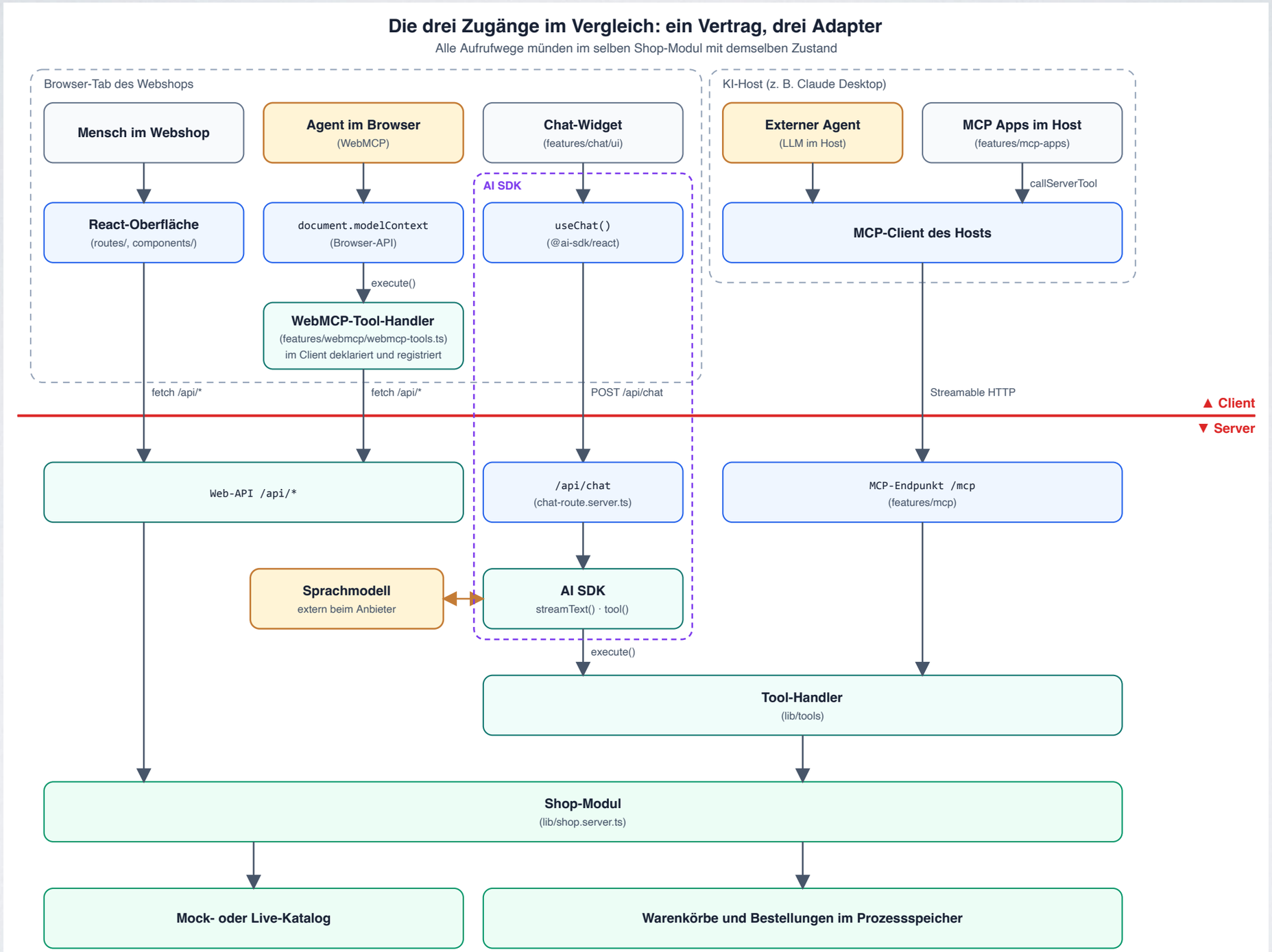
WebMCP im Webshop

Nur der Aufruf gegen unseren Server ist ein echter Netzwerk-Request



Merke: Das Sprachmodell steckt im Browser-Agent, die Tools stecken in der geöffneten Seite — beides ausserhalb unseres Servers.
Nur der Aufruf der Tools gegen unsere Web-API ist ein echter Netzwerk-Request; der Aufruf vom Agent zu den Tools bleibt im Browser.

Alles zusammen im Demo



Pfeile zeigen Aufrufwege, Rückgaben sind weggelassen. **Rote Linie:** Grenze zwischen Client und Server, jeder Pfeil darüber ist ein HTTP-Aufruf.
Gelb hinterlegte Boxen: hier entscheidet ein Sprachmodell, welches Tool mit welchen Argumenten aufgerufen wird. **Violett** umrahmt: vom AI SDK abstrahierter Chat-Pfad.

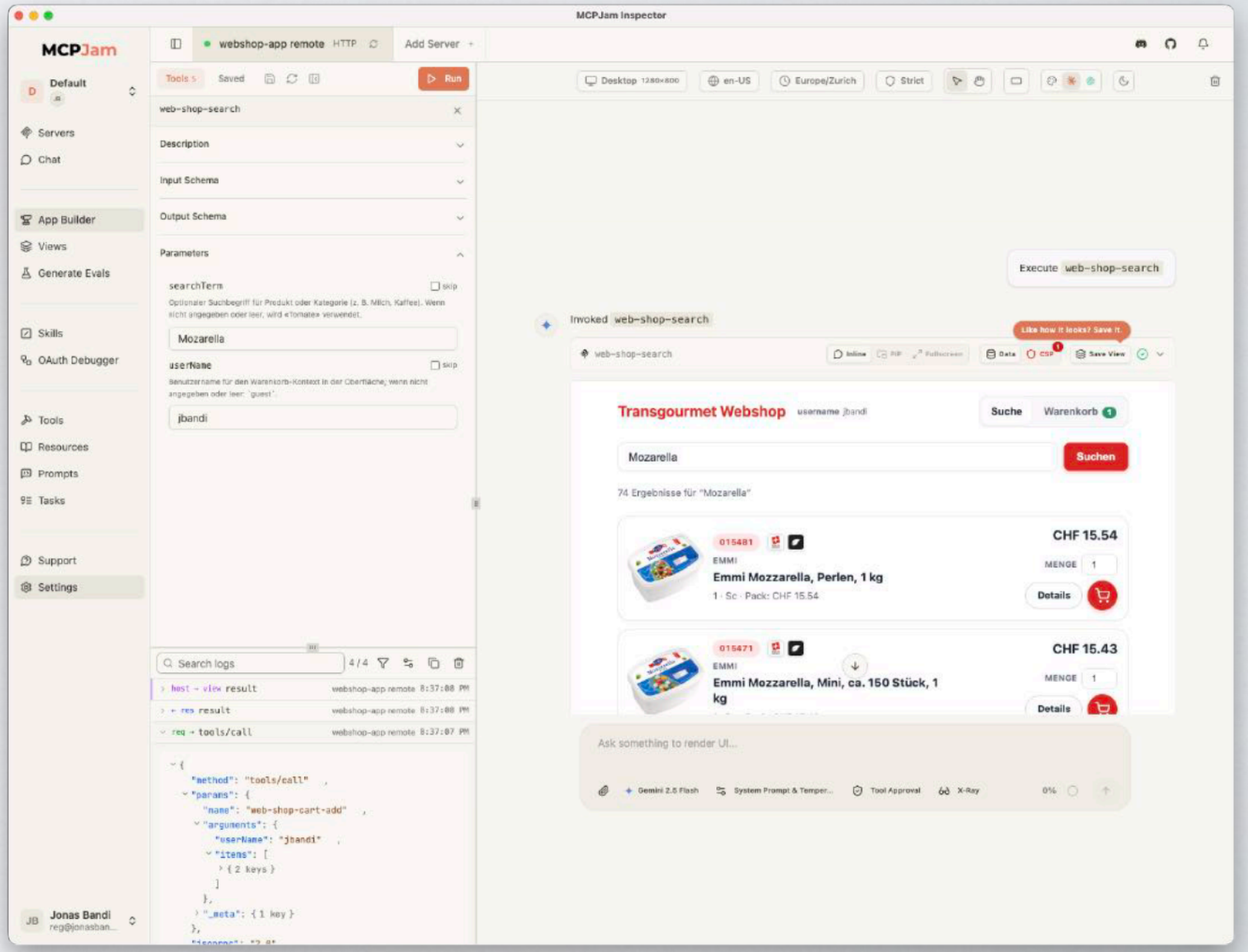
Testing & Development

- MCP Inspector:
  https://modelcontextprotocol.io/docs/tools/inspector

- MCP Jam Inspector
  https://www.mcpjam.com/

- Frameworks
  https://github.com/alpic-ai/skybridge

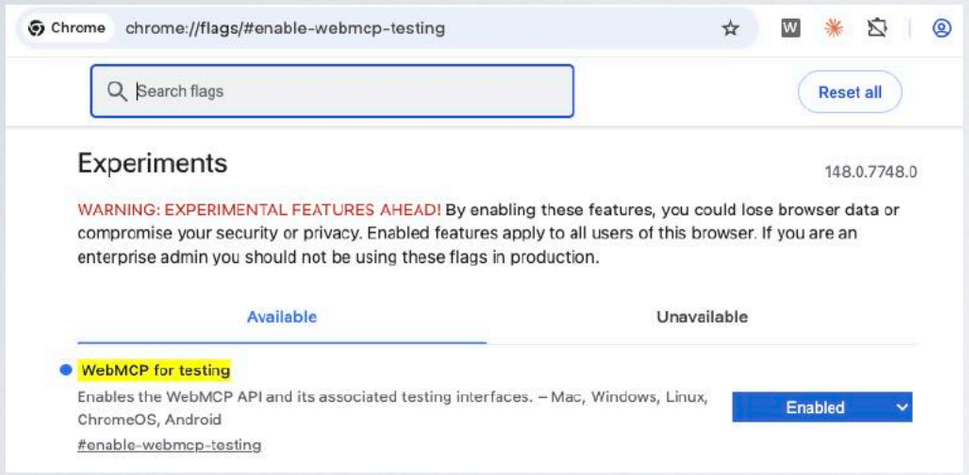
- Claude and ChatGPT
  `ngrok` or **`cloudflared tunnel`**



Postman also supports mcp and mcp apps.

Chapter 3: Agent controlled App

Using the experimental Chrome feature:



chrome://flags/#enable-webmcp-testing

Imperative API

```
const modelContext = navigator.modelContext
modelContext.registerTool({
  name: 'searchProducts',
  description:
    'Suche nach passenden Produkten im Shop-Katalog fuer einen Suchbegriff.',
  inputSchema: searchProductsSchema,
  execute: createToolExecutor((params) =>
    executeSearchProducts(params, callbacks),
  ),
})

modelContext.registerTool({
  name: 'addArticleToCart',
  description:
    'Fuege einen Artikel ueber seine Artikelnummer in den Warenkorb ein.',
  inputSchema: addArticleToCartSchema,
  execute: createToolExecutor((params) =>
    executeAddArticleToCart(params, callbacks),
  ),
})
```

business functionality shared with client impl

business functionality shared with client impl

Claude - Chrome Web Store | Experiments | Le Petit Bistro | WebMCP dec

googlechromelabs.github.io/webmcp-tools/demos/french-bistro/

Finish update

Le Petit Bistro

TABLE RESERVATIONS

FULL NAME

e.g. Alexander Hamilton

PHONE NUMBER

(555) 000-0000

DATE

dd.mm.yyyy

TIME

--:--

GUESTS

2 People

SEATING PREFERENCE

Main Dining Room

SPECIAL REQUESTS

Allergies, anniversaries, high chair...

REQUEST RESERVATION

WebMCP - Model Context Tool Inspector

WebMCP Tools

description	inputSchema	name
Creates a confirmed dining reservation at Le Petit Bistro. Accepts customer details, timing, and seating preferences.	{ "type": "object", "properties": { "name": { "type": "string", "description": "Customer's full name (min 2 chars)" }, "phone": { "type": "string", "description": "Customer's phone number (min 10 digits)" }, "date": { "type": "string", "format": "date", "description": "Reservation date (YYYY-MM-DD). Must be today or future. (Dates MUST be provided in 'YYYY-MM-DD' format.)" }, "time": { "type": "string", "format": "^([01][0-9]|2[0-3]):[0-5][0-9]$", "description": "Reservation time (HH:MM)" }, "guests": { "type": "string", "oneOf": [{ "type": "string", "const": "1", "title": "1 Person" }, { "type": "string", "const": "2", "title": "2 People" }, { "type": "string", "const": "3", "title": "3 People" }, { "type": "string", "const": "4", "title": "4 People" }, { "type": "string", "const": "5", "title": "5 People" }, { "type": "string", "const": "6", "title": "6 People or more" }], "enum": ["1", "2", "3", "4", "5", "6"], "description": "Number of people dining. Must be a string value between '1' and '5', or '6' for parties of 6 or more." }, "seating": { "type": "string", "oneOf": [{ "type": "string", "const": "Main Dining", "title": "Main Dining Room" }, { "type": "string", "const": "Terrace", "title": "Terrace (Outdoor)" }, { "type": "string", "const": "Private Booth", "title": "Private Booth" }, {	book_table_le_petit_bistro

Interact with the Page

User Prompt

Suche die Zutaten für Lasagne und lege den jeweils günstigsten Artikel in den Warenkorb.

Send | Reset | Set Gemini API key | Copy trace

Tool

"book_table_le_petit_bistro"

Input Arguments

```
{
 "name": "example_string",
 "phone": "example_string",
```

Execute Tool

Elements | Console | Sources | Network | Performance | Memory | Application | Security | Lighthouse

```
::before
<h2>Le Petit Bistro</h2>
<span class="subtitle">Table Reservations</span>
<form id="reservationForm" toolname="book_table_le_petit_bistro" tooldescription="Creates a confirmed dining reservation at Le Petit Bistro. Accepts customer details, timing, and seating preferences." novalidate>
  <div class="form-group">
    <label for="name">Full Name</label> == $0
    <input type="text" id="name" name="name" placeholder="e.g. Alexander Hamilton" required minlength="2" toolparamdescription="Customer's full name (min 2 chars)">
    <span class="error-msg">Please enter a valid name (at least 2 characters).</span>
  </div>
```

html | body | div.booking-container | form#reservationForm | div.form-group | label

Styles | Computed | Layout

Filter :hov .cls

```
element.style {
}
label {                                   style.css:85
  display: block;
  margin-bottom: 8px;
  color: var(--black);
  font-family: 'Segoe UI', sans-serif;
  font-size: 0.75em;
  font-weight: 700;
  text-transform: uppercase;
```

Declarative API

https://googlechromelabs.github.io/webmcp-tools/demos/french-bistro/

Questions?

Have fun with MCP!

Slides & Code:
https://github.com/ivorycode/mcp-baernerjs-2026/





Jonas Bandi
JavaScript / Angular / React / Vue / Vaadin
Schulung / Beratung / Coaching / Reviews
jonas.bandi@ivorycode.com